Machine Intelligence 1 Tutorial

04 Gradient methods, Regularization and Validation

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

Gradient methods

- The need for gradient-based optimization
- "Scope" of the gradient
- Learning rate schedules
- The Conjugate gradient method

Regularization

- L2 regularization: weight decay
- L1 regularization

Validation

- Overfitting vs. Underfitting
- Improving how we estimate $\widehat{E}^G$

Additional Material

- Data, Model and Objective
- Calculating the gradient
- Finding minima
- Represent the cost, gradient and Hessian in matrix form
- Solution with regularization
- A non-linearly separable problem
- Nonlinear basis function

Gradient methods

Todays setting - a linear regression problem

- Data:

  observations $x \in \mathbb{R}$

  label $y_T \in \mathbb{R}$

  Dataset $\left\{ \left(x^{(1)}, y_T^{(1)}\right), \ldots, \left(x^{(\alpha)}, y_T^{(\alpha)}\right), \ldots, \left(x^{(p)}, y_T^{(p)}\right) \right\}$
- Model: linear neuron $y(x; \underline{\mathbf{w}}) = w_0 + w_1 x = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}$
- Cost function: Quadratic error $E^T = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{1}{2} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)} \right)^2$
- The gradient of chosen cost function
  $\underline{\mathbf{g}} = \frac{1}{p} \sum_{\alpha=1}^{p} \left(\underline{\mathbf{x}}^{(\alpha)} \, \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)} \right) = \frac{1}{p} \left(\underline{\mathbf{X}} \, \underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}} \, \underline{\mathbf{y}}_{\text{True}}^\top \right).$
- The Hessian matrix $\underline{\mathbf{H}} = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)} \, \underline{\mathbf{x}}^{(\alpha)\top} = \frac{1}{p} \underline{\mathbf{X}\mathbf{X}}^\top$.
- Want $\underline{\mathbf{w}}^*$ s.t. $\underline{\mathbf{g}}(\underline{\mathbf{w}}^*) = 0$.

Q: *For centered data and* $\underline{\mathbf{H}} = \frac{1}{p} \underline{\mathbf{X}\mathbf{X}}^\top$, *what can we say about the type of the optimum?*

Todays setting - a linear regression problem

- Data:

  observations $x \in \mathbb{R}$

  label $y_T \in \mathbb{R}$

  Dataset $\left\{ \left(x^{(1)}, y_T^{(1)}\right), \ldots, \left(x^{(\alpha)}, y_T^{(\alpha)}\right), \ldots, \left(x^{(p)}, y_T^{(p)}\right) \right\}$
- Model: linear neuron $y(x; \underline{\mathbf{w}}) = w_0 + w_1 x = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}$
- Cost function: Quadratic error $E^T = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{1}{2} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)} \right)^2$
- The gradient of chosen cost function
  $\underline{\mathbf{g}} = \frac{1}{p} \sum_{\alpha=1}^{p} \left(\underline{\mathbf{x}}^{(\alpha)} \, \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)} \right) = \frac{1}{p} \left(\underline{\mathbf{X}} \, \underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}} \, \underline{\mathbf{y}}_{\text{True}}^\top \right).$
- The Hessian matrix $\underline{\mathbf{H}} = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)} \, \underline{\mathbf{x}}^{(\alpha)\top} = \frac{1}{p} \underline{\mathbf{X}} \underline{\mathbf{X}}^\top$.
- Want $\underline{\mathbf{w}}^*$ s.t. $\underline{\mathbf{g}}(\underline{\mathbf{w}}^*) = 0$.

Q: *For centered data and* $\underline{\mathbf{H}} = \frac{1}{p} \underline{\mathbf{X}} \underline{\mathbf{X}}^\top$*, what can we say about the type of the optimum?*

In that case, $\underline{\mathbf{H}} = \underline{\mathbf{C}}$ with $\underline{\mathbf{C}}$ the covariance matrix of the dataset $\underline{\mathbf{X}} = (\underline{\mathbf{x}}^{(1)}, \underline{\mathbf{x}}^{(2)}, \ldots \underline{\mathbf{x}}^{(p)})$, which is positive semi-definite.

Q: *In what case(s) do we avoid an analytical solution?*

Q: *In what case(s) do we avoid an analytical solution?*

- We cannot fit all the data into memory.
- $$\underline{\mathbf{g}} = \frac{1}{p}\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top\right) = 0 \Leftrightarrow \underline{\mathbf{w}}^* = (\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top)^{-1}\underline{\mathbf{X}}\,\underline{\mathbf{y}}_{True}^\top$$

  but $\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top$ is not invertible
- no closed-form solution for our model (e.g. MLP with non-linearities)
- Computing the Hessian is too costly. Newton's method also not applicable.
- Non-stationary data. We cannot easily adapt to changes over time.

Alternative: iterative gradient-based optimization

$$\underline{\mathbf{w}}_{t+1} = \underline{\mathbf{w}}_t - \underbrace{\eta_t \left.\frac{\partial E^T}{\partial \underline{\mathbf{w}}}\right|_{\underline{\mathbf{w}}_t}}_{\text{update } \Delta\underline{\mathbf{w}}}$$

with $\eta_t \in \mathbb{R}^+$, then $E^T_{[\underline{\mathbf{w}}_{t+1}]} \leq E^T_{[\underline{\mathbf{w}}_t]}$.

Alternative: iterative gradient-based optimization

$$\underline{\mathbf{w}}_{t+1} = \underline{\mathbf{w}}_t - \underbrace{\eta_t \left.\frac{\partial E^T}{\partial \underline{\mathbf{w}}}\right|_{\underline{\mathbf{w}}_t}}_{\text{update } \Delta\underline{\mathbf{w}}}$$

with $\eta_t \in \mathbb{R}^+$, then $E^T_{[\underline{\mathbf{w}}_{t+1}]} \leq E^T_{[\underline{\mathbf{w}}_t]}$.

The behavior of gradient descent can be controlled by modulating

- The "scope" of the gradient. What do we use for deciding on the direction of the update?
- The learning rate schedule. How big of a step do we take in the direction of the gradient?

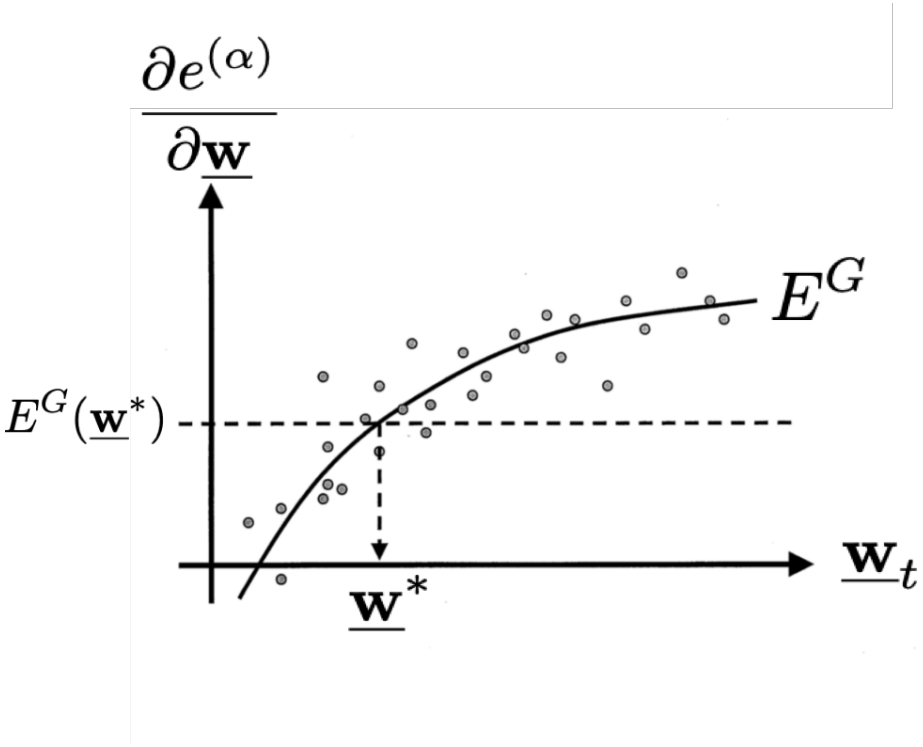
"Scope" of the gradient

batch	$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e^{(\alpha)}}{\partial \underline{\mathbf{w}}}$	
mini-batch	$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} \approx \frac{1}{\lvert\mathcal{M}\rvert}\sum_{\beta\in\mathcal{M}}^{\lvert\mathcal{M}\rvert} \frac{\partial e^{(\beta)}}{\partial \underline{\mathbf{w}}}$ $\lvert\mathcal{M}\rvert \ll p$ where $\mathcal{M}$ is a set of indices sampled randomly from $\{1,\ldots,p\}$	noisy estimate of E^T, becomes less noisy with larger $\lvert\mathcal{M}\rvert$
online	$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} \approx \frac{\partial e^{(\beta)}}{\partial \underline{\mathbf{w}}}$ a single random data point $(\underline{\mathbf{x}}^{(\beta)}, \underline{\mathbf{y}}_T^{(\beta)})$	very noisy estimate of E^T

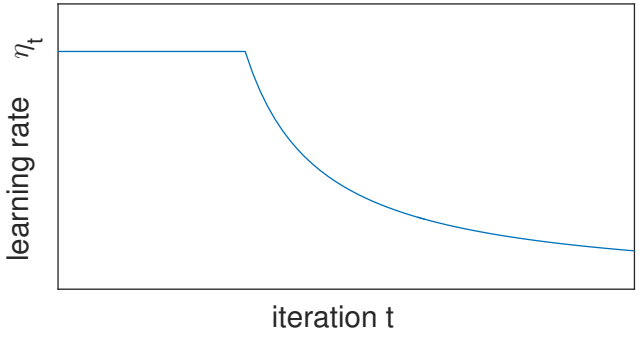
Q: *Which of these can handle non-stationary data?*

Robbins & Monro condition

(a) Stochastic Approximation

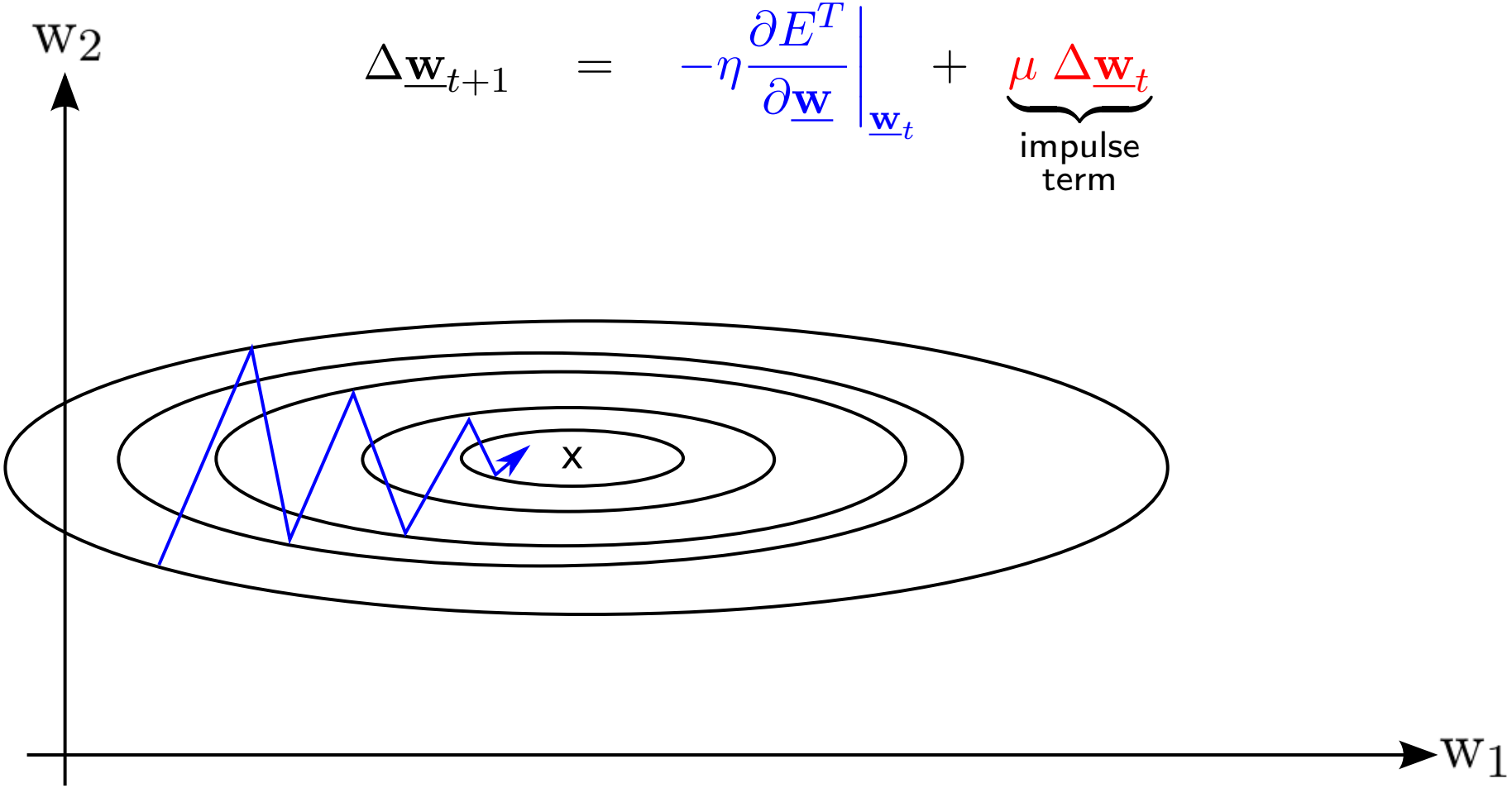


(b) Decaying learning rate



Impulse terms

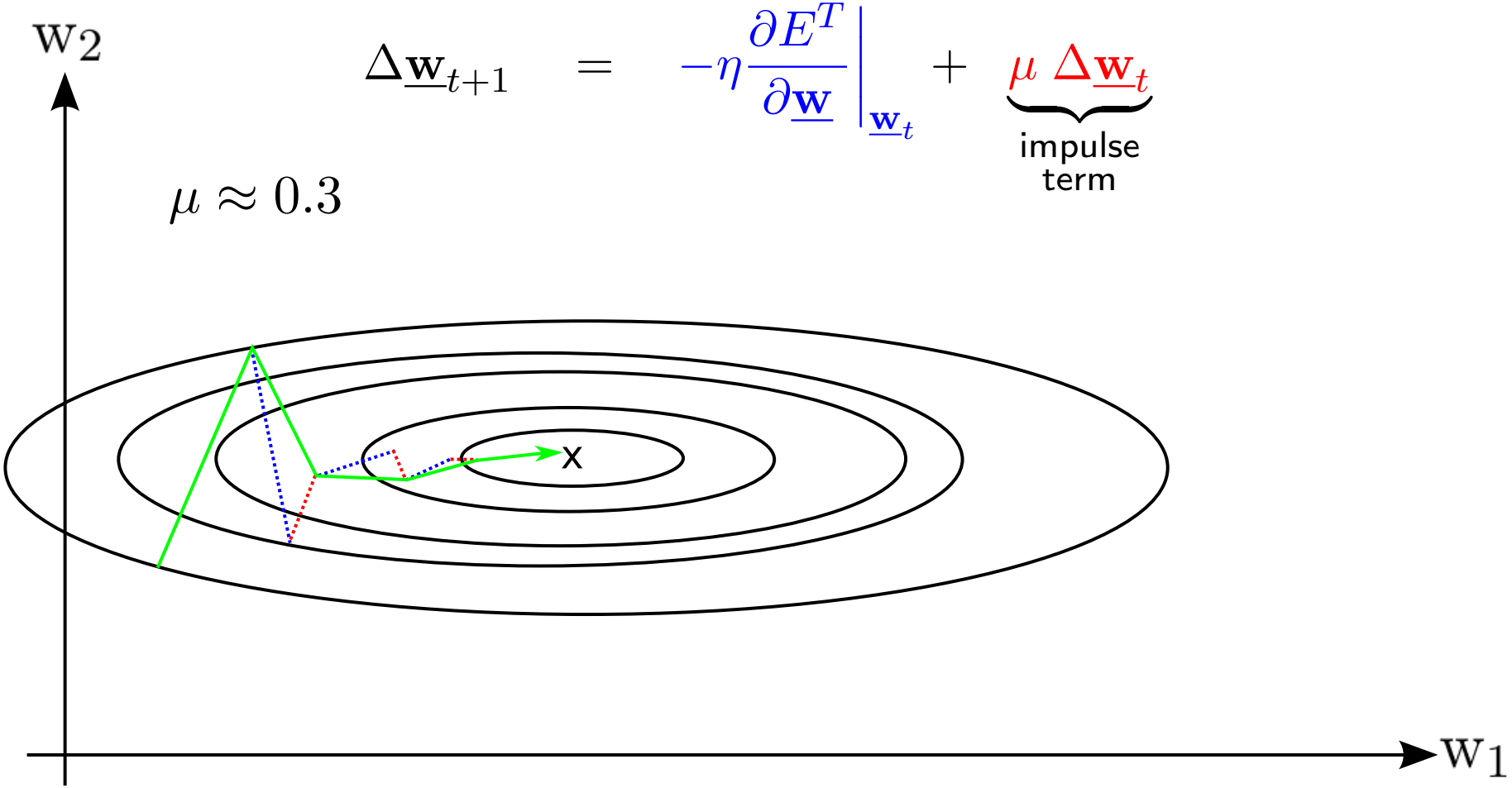
$$\Delta \underline{\mathbf{w}}_{t+1} = -\eta \frac{\partial E^T}{\partial \underline{\mathbf{w}}}\bigg|_{\underline{\mathbf{w}}_t} + \underbrace{\mu\, \Delta \underline{\mathbf{w}}_t}_{\text{impulse term}}$$



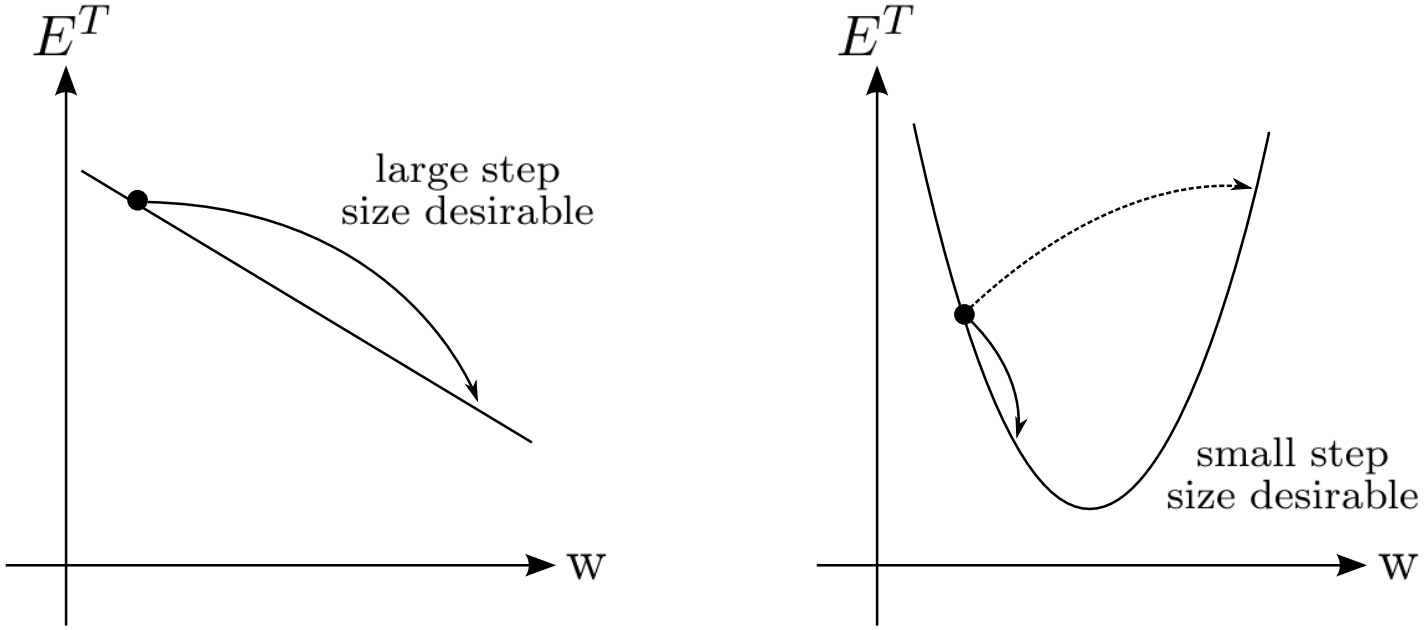
Impulse terms

$$\Delta \underline{\mathbf{w}}_{t+1} = -\eta \frac{\partial E^T}{\partial \underline{\mathbf{w}}}\bigg|_{\underline{\mathbf{w}}_t} + \underbrace{\mu\, \Delta \underline{\mathbf{w}}_t}_{\text{impulse term}}$$

$\mu \approx 0.3$



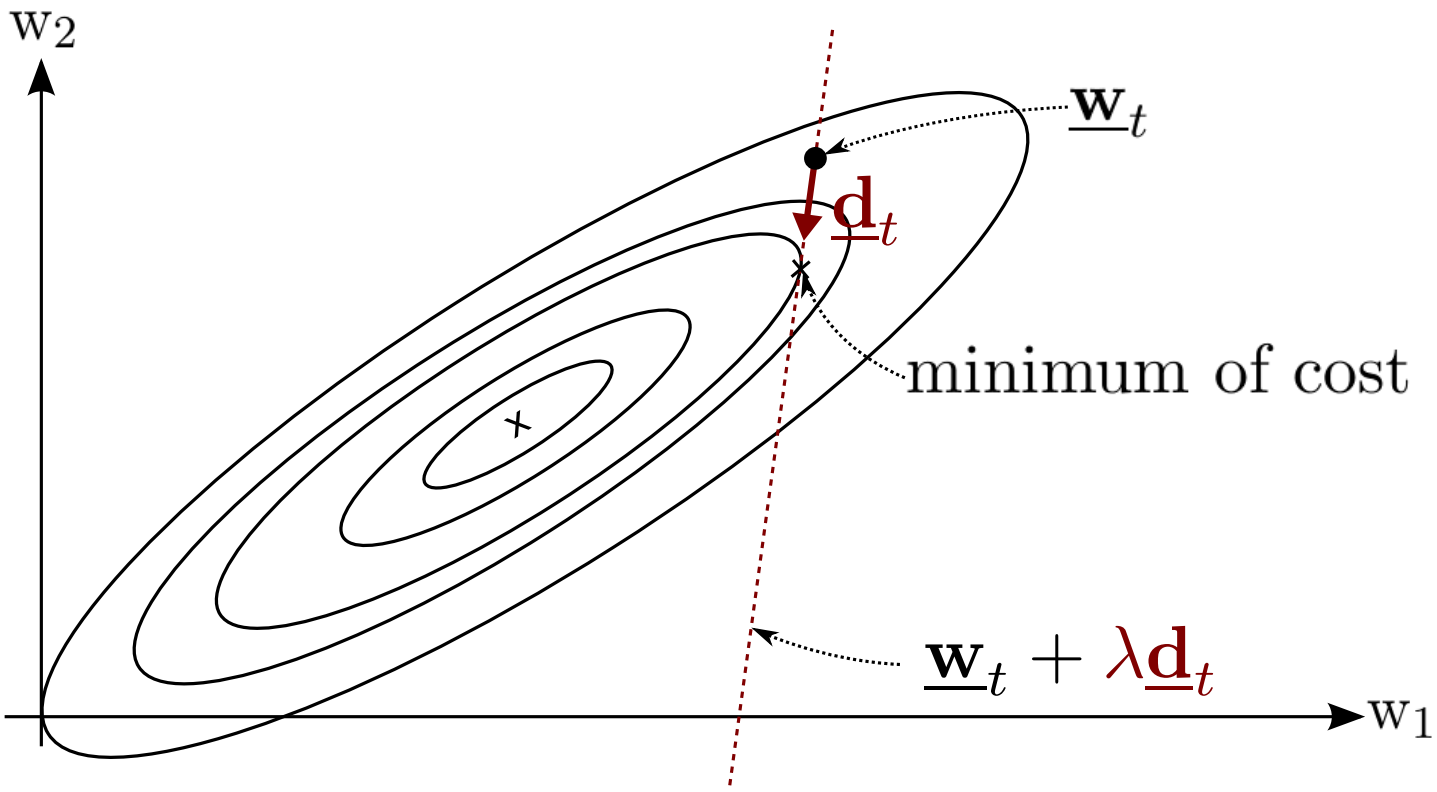
Adaptive step size



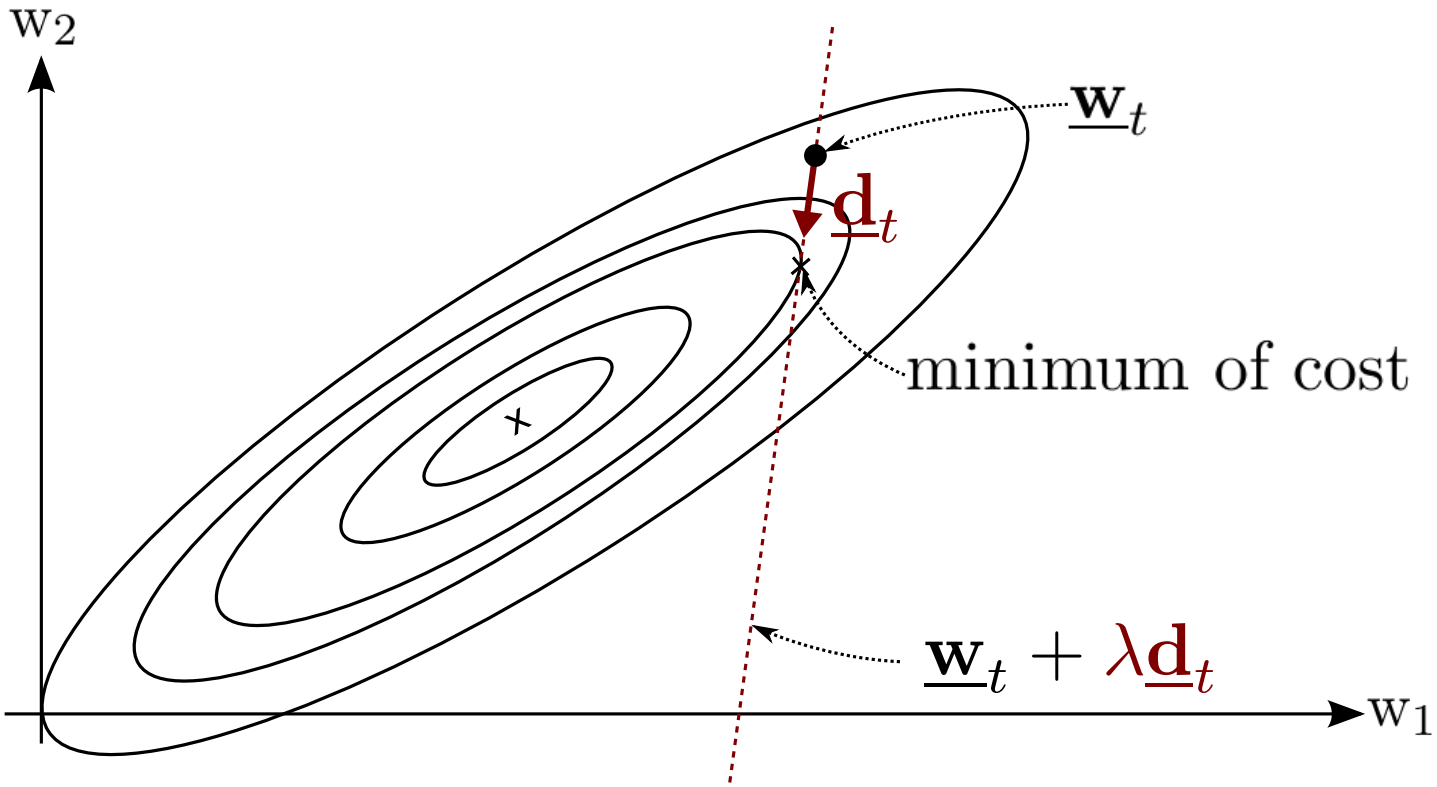
$$\eta_{t+1} = \begin{cases} \rho\eta_t, & \text{if } \Delta E^T < 0, \quad \text{increase step size, if } E^T \downarrow \\ \delta\eta_t, & \text{if } \Delta E^T > 0, \quad \text{decrease step size, if } E^T \uparrow \end{cases}$$

typical values: $\rho = 1.1, \delta = 0.5$

Line search

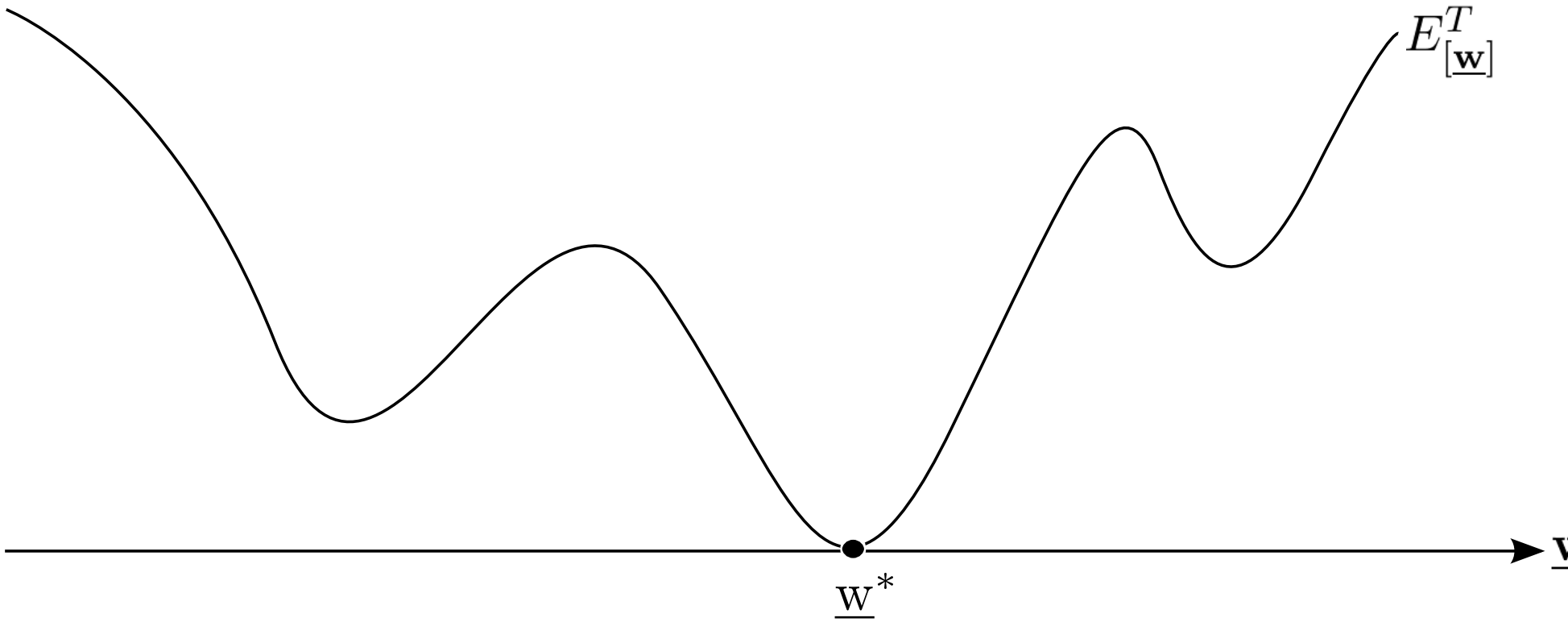

w_2
$\underline{\mathbf{w}}_t$
$\underline{\mathbf{d}}_t$
minimum of cost
$\underline{\mathbf{w}}_t + \lambda \underline{\mathbf{d}}_t$
w_1

Parabolic interpolation - Blackboard



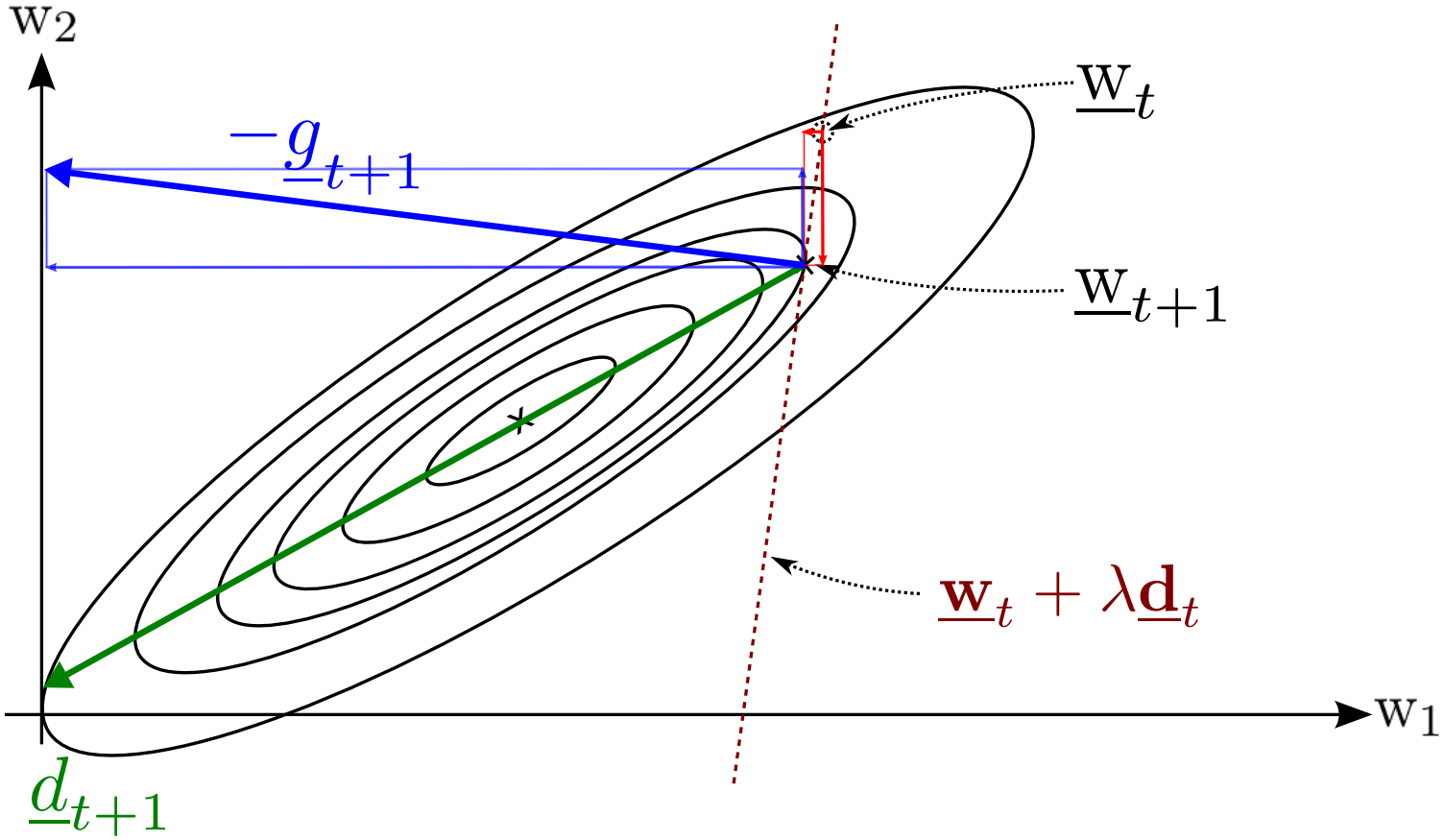
Hint: Corresponding pseudo-code of algorithms and optimal step sizes - see lecture.

Line search



Q: *What are potential pitfalls of parabolic interpolation?*

Limitations of line search



Conjugate gradient method

$$\underline{\mathbf{w}}_{t+1} \quad = \quad \underline{\mathbf{w}}_t + \eta_t \, \underline{\mathbf{d}}_t$$

Don't undo progress of previous search directions $\underline{\mathbf{d}}_t$, $\underline{\mathbf{d}}_{t-1}$.
In other words:
The component of the gradient parallel to the old direction $\underline{\mathbf{d}}_t$ should remain zero along the new direction $\underline{\mathbf{d}}_{t+1}$

Conjugate gradient method

$$\underline{\mathbf{w}}_{t+1} \quad = \quad \underline{\mathbf{w}}_t + \eta_t \, \underline{\mathbf{d}}_t$$

Don't undo progress of previous search directions $\underline{\mathbf{d}}_t$, $\underline{\mathbf{d}}_{t-1}$.
In other words:
The component of the gradient parallel to the old direction $\underline{\mathbf{d}}_t$ should remain zero along the new direction $\underline{\mathbf{d}}_{t+1}$

Initialization: $\underline{\mathbf{w}}, \quad \underline{\mathbf{d}} = -\underline{\mathbf{g}}$

while *stopping criterion not fulfilled* **do**

Minimize E^T along $\underline{\mathbf{d}}$ using line-search $\rightarrow$ new $\underline{\mathbf{w}}$

Calculate the new conjugate direction $\rightarrow$ new $\underline{\mathbf{d}}$

end

Adaptive momentum: the Polak-Ribiere rule

i.e. how to calculate $\underline{\mathbf{d}}_{t+1}$

see lecture - Polak-Ribiere rule

Conjugate gradient descent algorithm

Initialization: $\underline{\mathbf{w}}, \quad \underline{\mathbf{d}} = -\underline{\mathbf{g}}$

while *stopping criterion not fulfilled* **do**

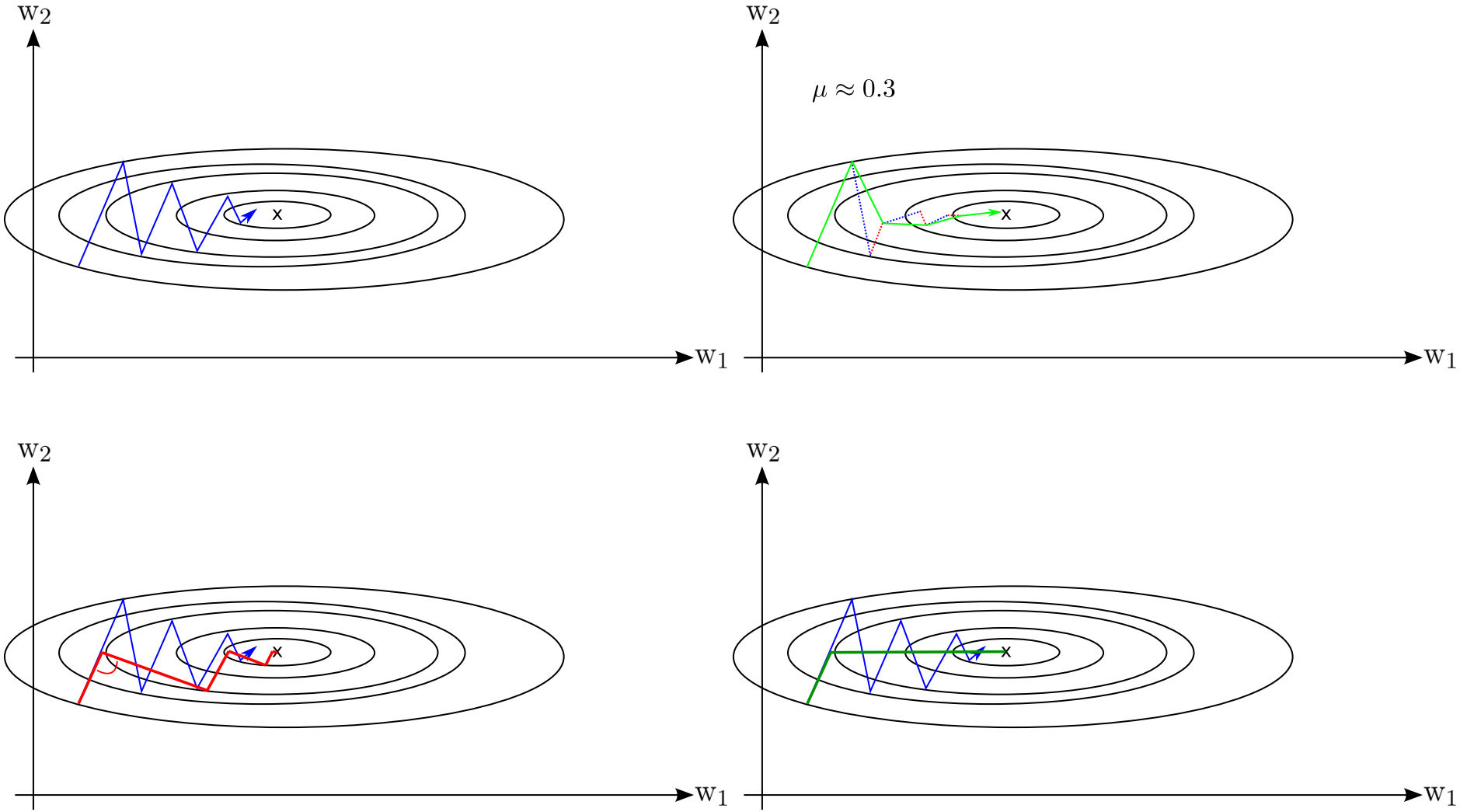
Minimize E^T along $\underline{\mathbf{d}}$ using line-search $\rightarrow$ new $\underline{\mathbf{w}}$

Calculate the new conjugate direction $\rightarrow$ new $\underline{\mathbf{d}}$

end

- Subsequent search directions lose conjugacy
  - search direction should be reset to the steepest descent direction at least every N iterations (in practice apply heuristic, e.g. reset every 5 steps)

Comparison



Where are we?

Gradient methods

Regularization

Validation

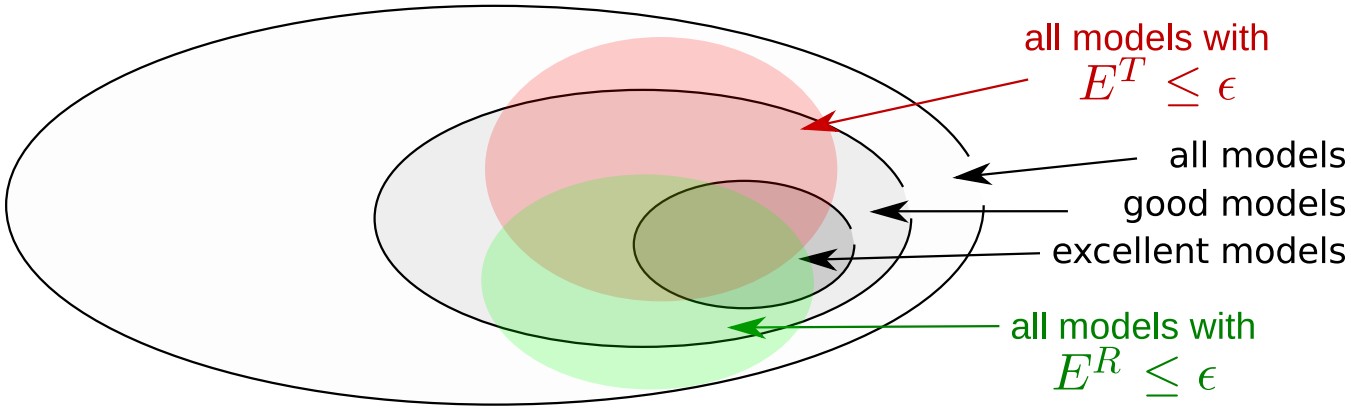
Additional Material

Regularization

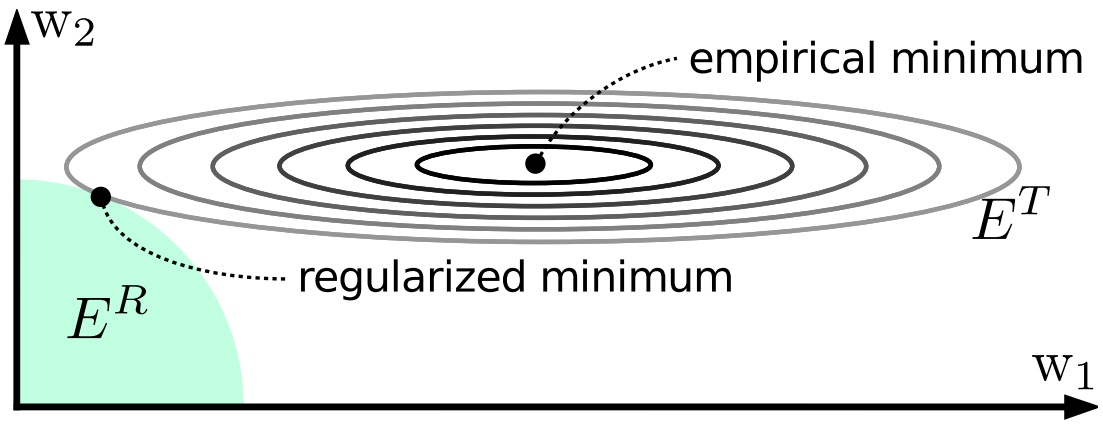
Risk function

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training}\\ \text{error}\\ \text{e.g. quadratic cost,}\\ \text{cross-entropy,...}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization}\\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

- $E^R_{[\underline{\mathbf{w}}]}$: prior knowledge of solution
- λ : regularization parameter



Regularization example: L_2 regularization (weight decay)



$$E^R_{[\underline{\mathbf{w}}]} = \frac{1}{2} \sum_{(i,j,v',v)} \left(w_{ij}^{v'v}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}}$$

Using vector notation to compute the gradient:

$$\frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} = \frac{1}{\cancel{2}} \cdot \cancel{2} \cdot \underline{\mathbf{w}} = \underline{\mathbf{w}}$$

Regularized linear neuron for regression: analytical solution

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training error} \\ \text{e.g. quadratic cost}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization} \\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

Gradient:

$$\underline{\mathbf{g}}^R := \frac{\partial}{\partial \underline{\mathbf{w}}} R_{[\underline{\mathbf{w}}]} = \frac{\partial}{\partial \underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} + \lambda \frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]}$$

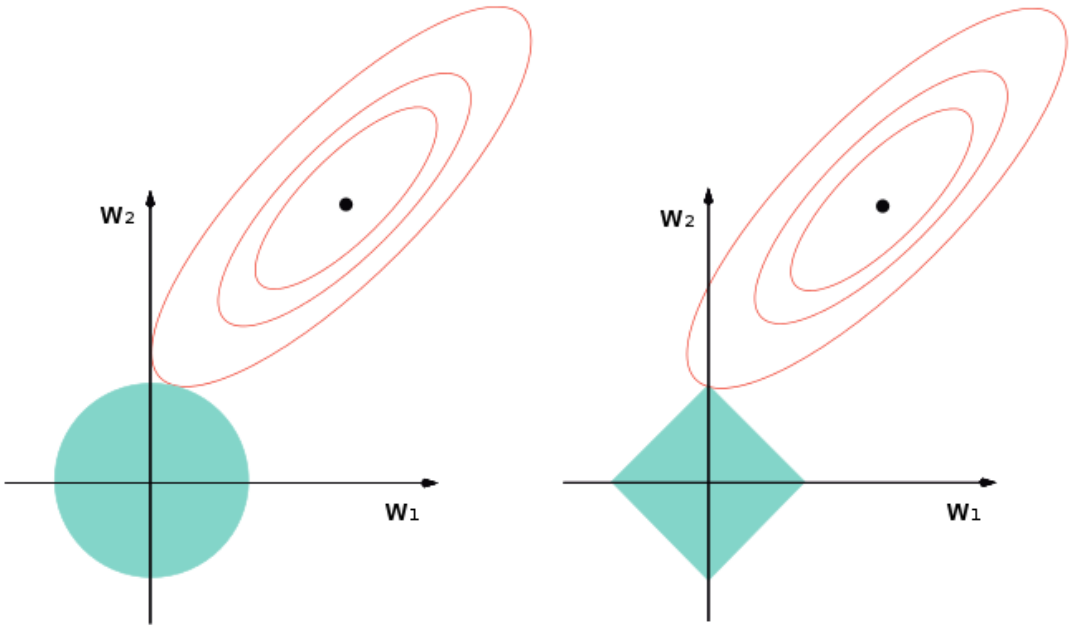
$$= \frac{1}{p} \left(\underline{\mathbf{X}}\, \underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}}\, \underline{\mathbf{y}}^\top_{\text{True}} \right) + \underline{\mathbf{I}}_N \, \lambda \, \underline{\mathbf{w}}$$

$$\iff \underline{\mathbf{g}}^R = \underline{\mathbf{0}}$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\, \underline{\mathbf{X}}^\top + p\lambda \, \underline{\mathbf{I}}_N \right)}_{\substack{\text{positive definite and} \\ \text{therefore invertible}}}^{-1} \underline{\mathbf{X}}\, \underline{\mathbf{y}}^\top_{\text{True}}$$

L1 regularization

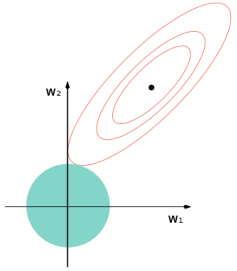
$$E^R_{[\underline{\mathbf{w}}]} = \sum_{(i,j,v',v)} |w_{ij}^{v'v}|^1$$



L_2 *weight decay* L_1 *lasso*

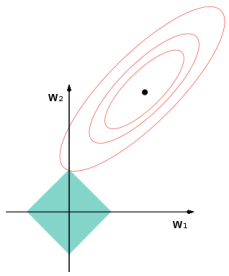
"Lasso" stands for "**l**east **a**bsolute **s**hrinkage and **s**election **o**perator".

L2 vs. L1 regularization



$L2$, weight decay:

- "standard" regularizer
- differentiable (simple integration into gradient descent procedure)
- robust to noise (results in a "distributed" model, redundancies in weights allowed)
- non-sparse



$L1$, Lasso, "sparsify":

- sparse (good for interpretable models, e.g. medical applications)
- non-differentiable at $w_i = 0 \forall i$ (but workaround via subgradient/least angle regression)
- less redundancies between weights $\rightsquigarrow$ more prone to noise

Where are we?

Gradient methods

Regularization

Validation

Additional Material

Validation

Recall: performance measure, a criterion for *model selection*.

$$E^G_{[\underline{\mathbf{w}}]} := \langle e \rangle_{y_T, \underline{\mathbf{x}}; \underline{\mathbf{w}}} = \iint d\underline{\mathbf{x}}\, dy_T\, P(y_T, \underline{\mathbf{x}})\, e(y_T, y(\underline{\mathbf{x}}; \underline{\mathbf{w}}))$$

Because $P(y_T, \underline{\mathbf{x}})$ is not known,
empirical average $E^T_{[\underline{\mathbf{w}}]}$ using the available training data:

$$\left\{ \left(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)} \right) \right\}, \alpha = 1, \ldots, p.$$

batch training error: $$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p} \sum_{\alpha=1}^{p} \underbrace{e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}} \right) \right)}_{e^{(\alpha)}}$$

or mini-batch and stochastic online approximations of $E^T_{[\underline{\mathbf{w}}]}$
not limited to scalar attributes y, y_T.

Q: *How well does E^T represent E^G?*

Training error alone is not enough

$\underline{\mathbf{w}}^* = \mathrm{argmin}_{\underline{\mathbf{w}}}\, E^T_{[\underline{\mathbf{w}}]}$ does not imply good generalization performance.

$$\text{small } E^T_{[\underline{\mathbf{w}}^*]} \quad \not\Rightarrow \quad \text{small } E^G_{[\underline{\mathbf{w}}^*]}$$

Expected:

$$E^T_{[\underline{\mathbf{w}}^*]} \quad < \quad E^G_{[\underline{\mathbf{w}}^*]}$$

Training error alone is not enough

$\underline{\mathbf{w}}^* = \operatorname{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$ does not imply good generalization performance.

$$\text{small } E^T_{[\underline{\mathbf{w}}^*]} \quad \not\Rightarrow \quad \text{small } E^G_{[\underline{\mathbf{w}}^*]}$$

Expected:

$$E^T_{[\underline{\mathbf{w}}^*]} \quad < \quad E^G_{[\underline{\mathbf{w}}^*]}$$

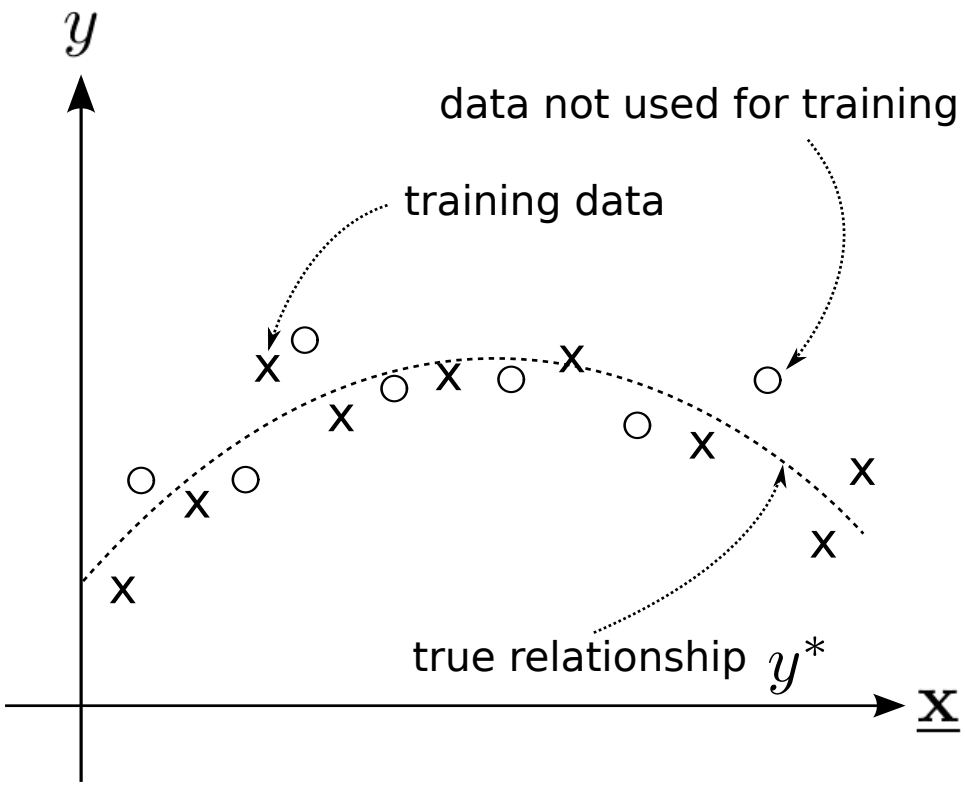
Validation: evaluate the generalization of a model (and its training procedure) by estimating

$$\widehat{E}^G_{[\underline{\mathbf{w}}]} \approx E^G_{[\underline{\mathbf{w}}]}$$

This implies that we need a set of data.

- $\{\underline{\mathbf{x}}^{(\alpha)}_{\text{train}}, y_T{}^{(\alpha)}_{\text{train}}\}^p_{\alpha=1}$ for model selection
- $\rightsquigarrow \underline{\mathbf{w}}^* = \operatorname{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$
- $\{\underline{\mathbf{x}}^{(\beta)}_{\text{val.}}, y_T{}^{(\beta)}_{\text{val.}}\}^q_{\beta=1}$ for validation
- $\rightsquigarrow \widehat{E}^G_{[\underline{\mathbf{w}}^*]}$

Overfitting vs. Underfitting

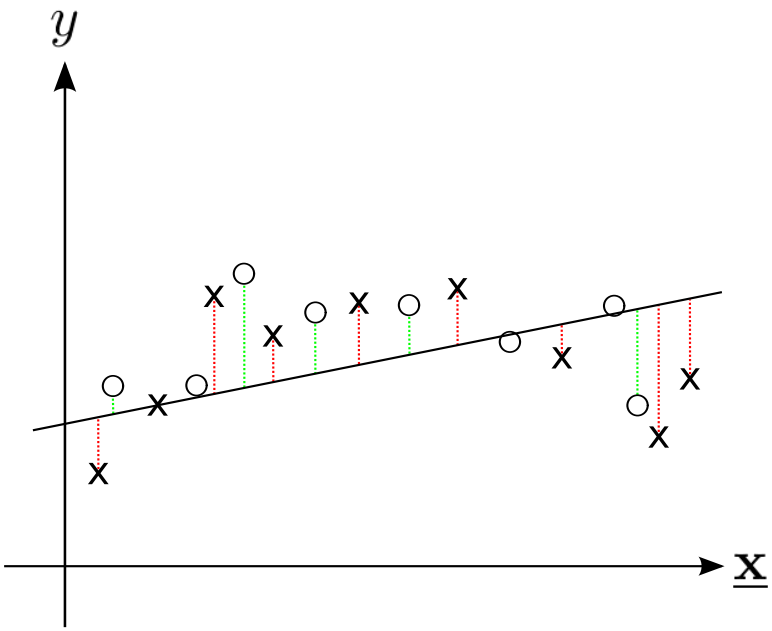


data generation:

e.g. $y_T = y^*_{(\underline{\mathbf{x}};\underline{\mathbf{w}})} + \underbrace{\eta}_{\text{noise, zero mean}}$

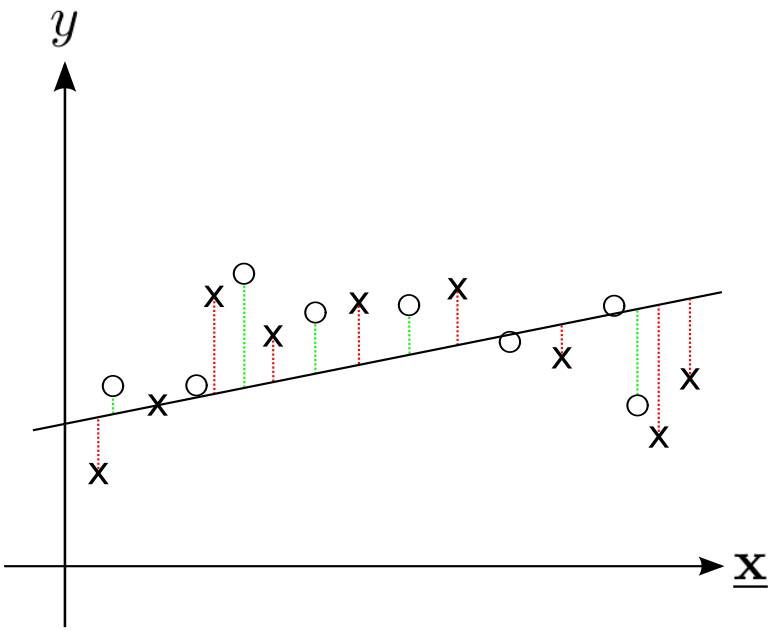
Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:



Overfitting vs. Underfitting

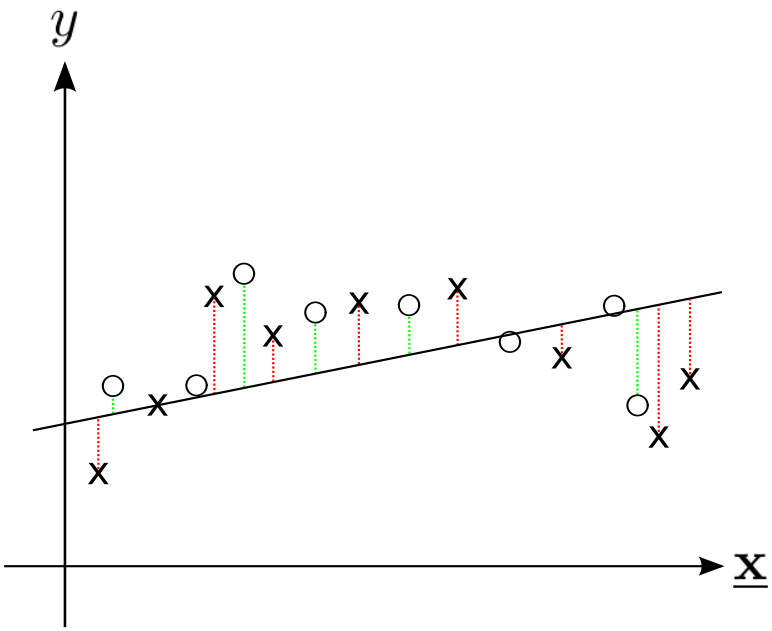
Compare E^T and $\widehat{E}^G$:

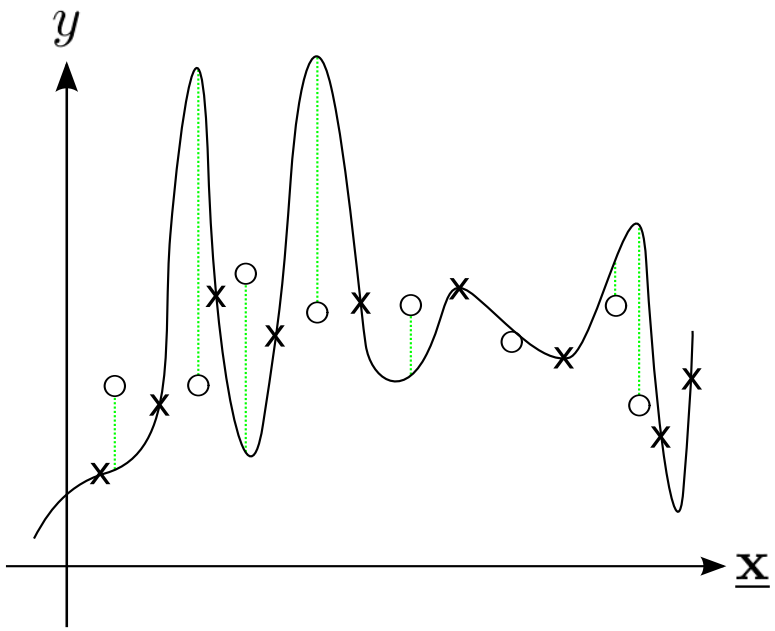


$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:

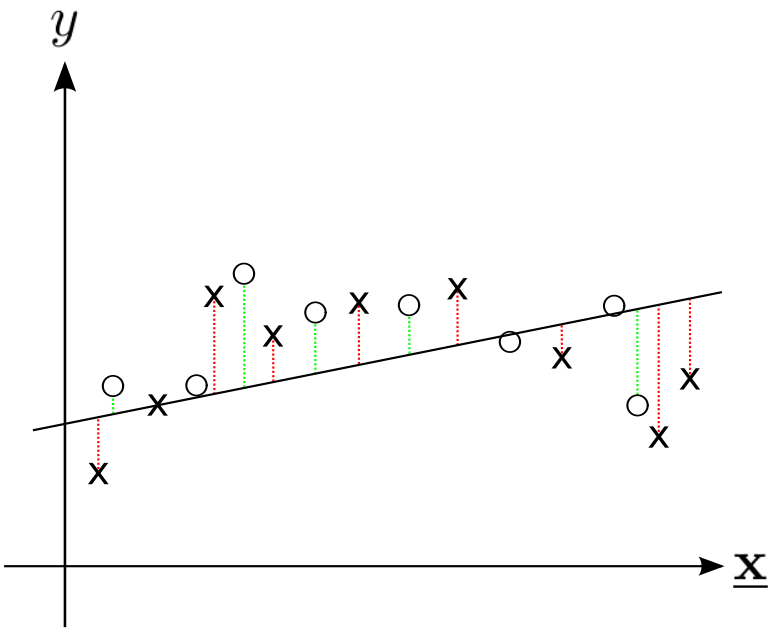


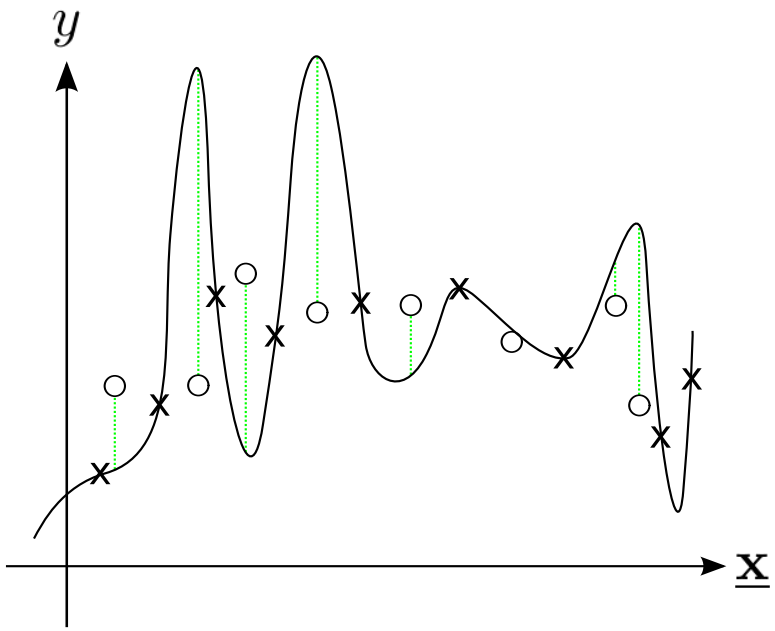


$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Overfitting vs. Underfitting

Compare E^T and $\widehat{E}^G$:

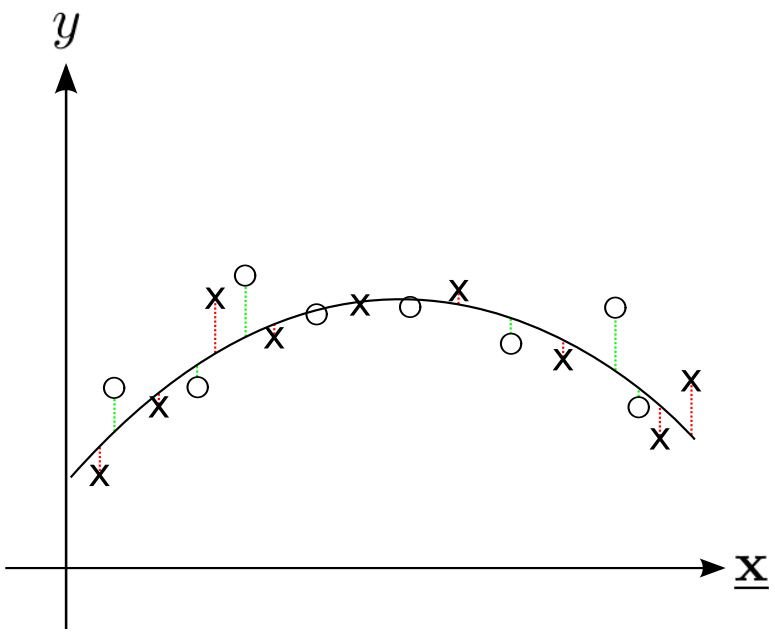




$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

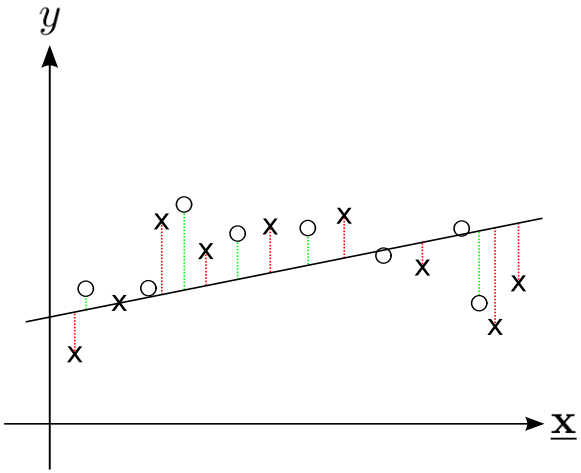
Well fitted model



Diagnostics

$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ small} \end{array}\right\} E^T \approx \widehat{E}^G$$

Underfitting



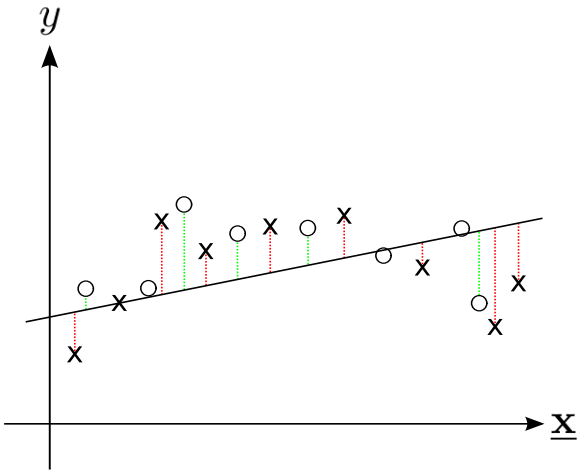
$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Q: *What is underfitting?*

- function complexity vs. model complexity
- the degrees of freedom in the model are too ...
- the model is insensitive to variations in the features

Q: *How would replacing a few training samples change the solution?*

Underfitting



$$\left.\begin{array}{l} E^T \text{ large} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \approx \widehat{E}^G$$

Q: *What is underfitting?*

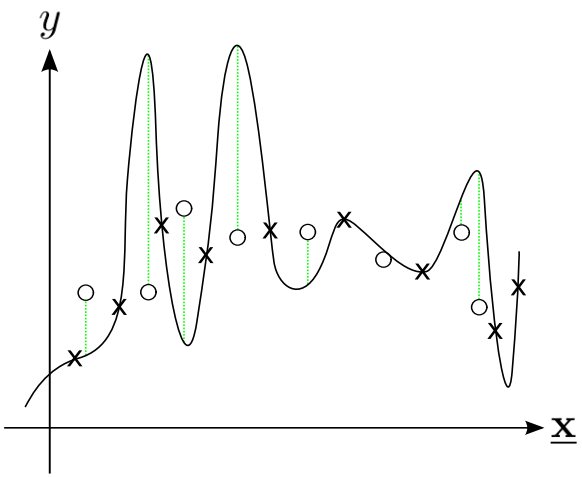
- function complexity vs. model complexity
- the degrees of freedom in the model are too ...
- the model is insensitive to variations in the features

Q: *How would replacing a few training samples change the solution?*

- Not much.

Overfitting

Q: *What is overfitting?*



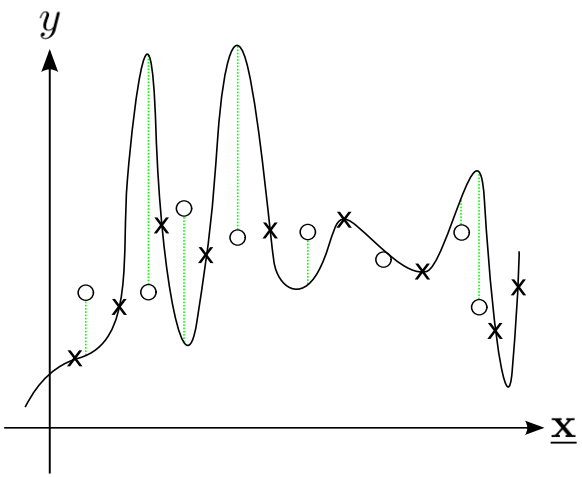
$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

- function complexity vs. model complexity
- the degrees of freedom in the model are...
- the model is over-sensitive to the noise component
- the model has "memorized" the training data instead of learning a representation for it.

Q: *How would replacing a few training samples change the solution?*

Overfitting

Q: *What is overfitting?*



$$\left.\begin{array}{l} E^T \text{ small} \\ \widehat{E}^G \text{ large} \end{array}\right\} E^T \ll \widehat{E}^G$$

- function complexity vs. model complexity
- the degrees of freedom in the model are...
- the model is over-sensitive to the noise component
- the model has "memorized" the training data instead of learning a representation for it.

Q: *How would replacing a few training samples change the solution?*

Dealing with overfitting/underfitting

	reduces $\rightarrow$	overfitting	underfitting
1	more training data		
2	larger reg. coeff. λ		
3	smaller reg. coeff. λ		
4	use a more complex model		
5	use a simpler model		
6	increase no. of hidden neurons in MLP		
7	add more hidden layers to MLP		
8	add features to the input $\underline{\mathbf{x}}$		
9	shuffle the data		

Dealing with overfitting/underfitting

	potentially reduces →	overfitting	underfitting
1	more training data	yes	no
2	larger regularization coeff. λ	yes	no
3	smaller regularization coeff. λ	no	yes
4	use more complex model	no	yes
5	use simpler model	yes	no
6	increase number of hidden neurons in MLP	no	yes
7	add more hidden layers to MLP	no	yes
8	add features to the input $\underline{\mathbf{x}}$	no	yes
9	shuffle the data	no	no
	more techniques?		

Different views of the bias-variance tradeoff

Remember: it's always a bias-variance <u>tradeoff</u>

Different views of the bias-variance tradeoff

Change the size of the training set p to identify potential overfitting vs. underfitting of the model:

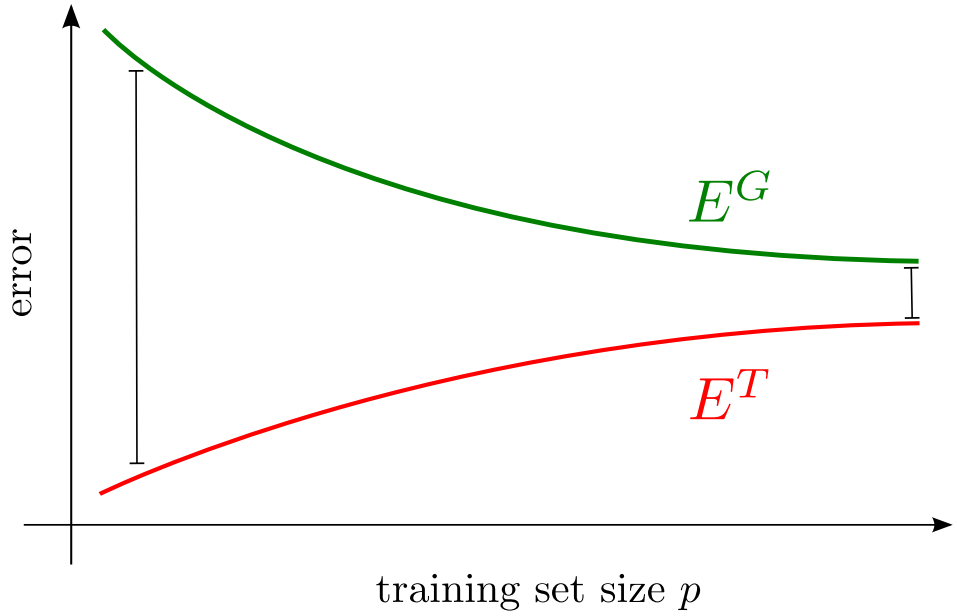
Q: *What* E^T *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* E^T *vs.* p *relationship do we expect for an overfitting model?*

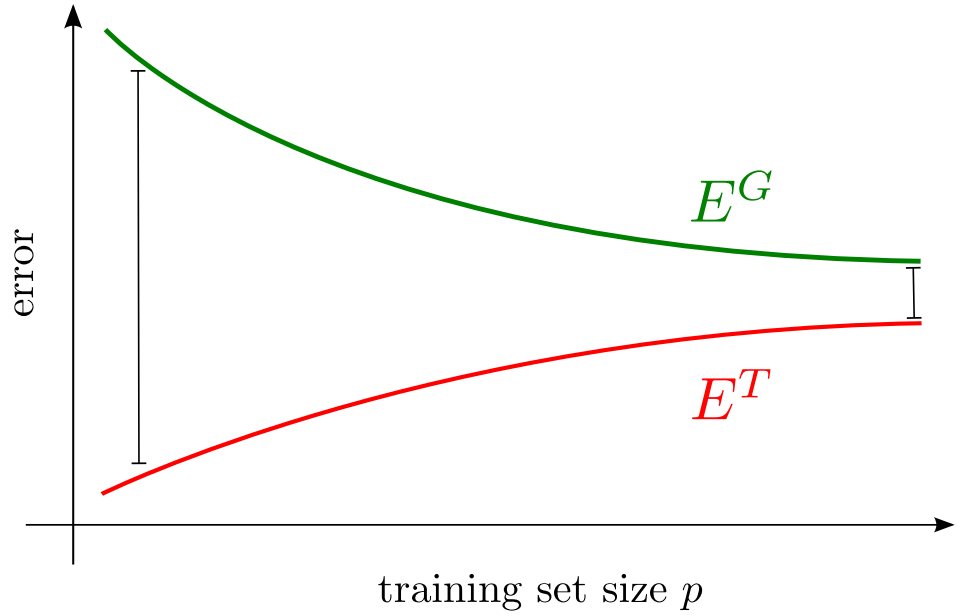
Q: *What* $\widehat{E}^G$ *vs.* p *relationship do we expect for an underfitting model?*

Q: *What* $\widehat{E}^G$ *vs.* p *relationship do we expect for an overfitting model?*

Different views of the bias-variance tradeoff

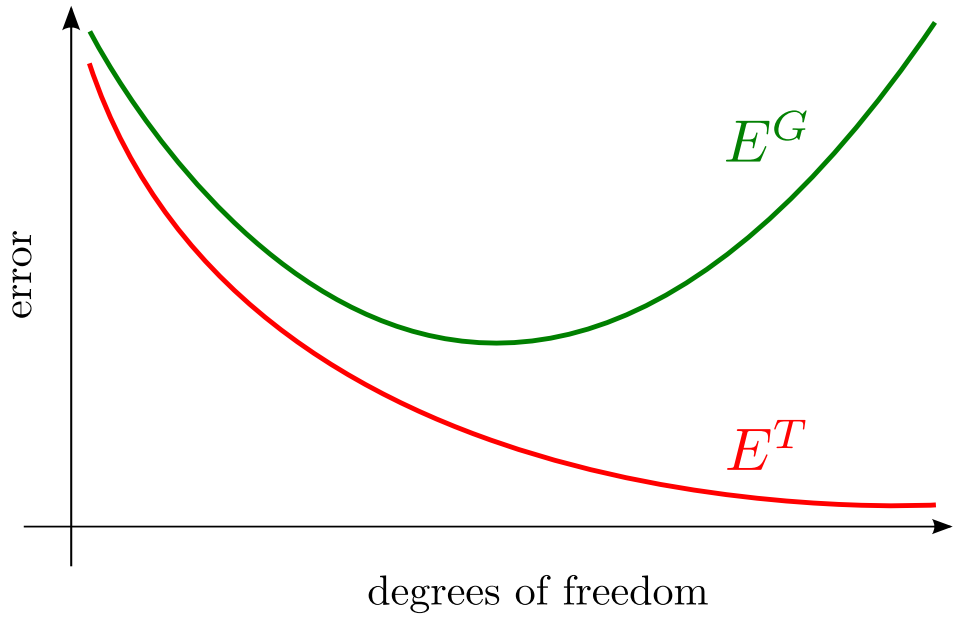


Different views of the bias-variance tradeoff

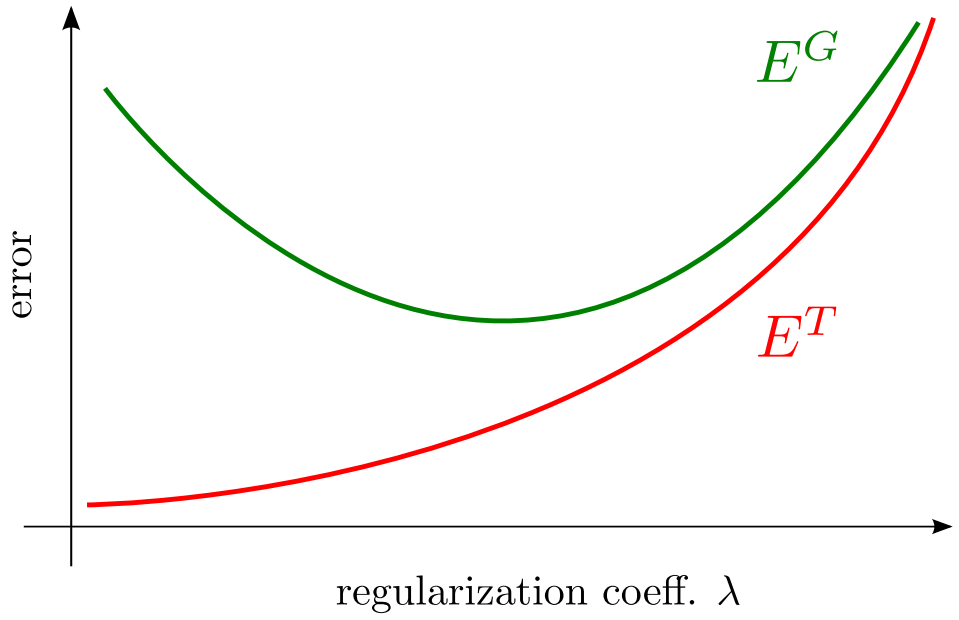


Q: *What is the benefit of looking at the bias-variance tradeoff as a function of p?*

Different views of the bias-variance tradeoff



Different views of the bias-variance tradeoff



Improving how we estimate $\widehat{E}^G$

Validation: Testset method

Testset method (with hyperparameters, e.g. regularization coeff. λ)

3-way split of the data.

1. training set: perform model selection for different values of λ
2. test set: select value of λ, which provides best prediction results on this set $\rightsquigarrow \lambda^{\text{opt}}$
3. validation set: estimate generalization performance of selected λ
   This is data that was not used for model selection or hyperparameter selection.

training set	test set	validation set

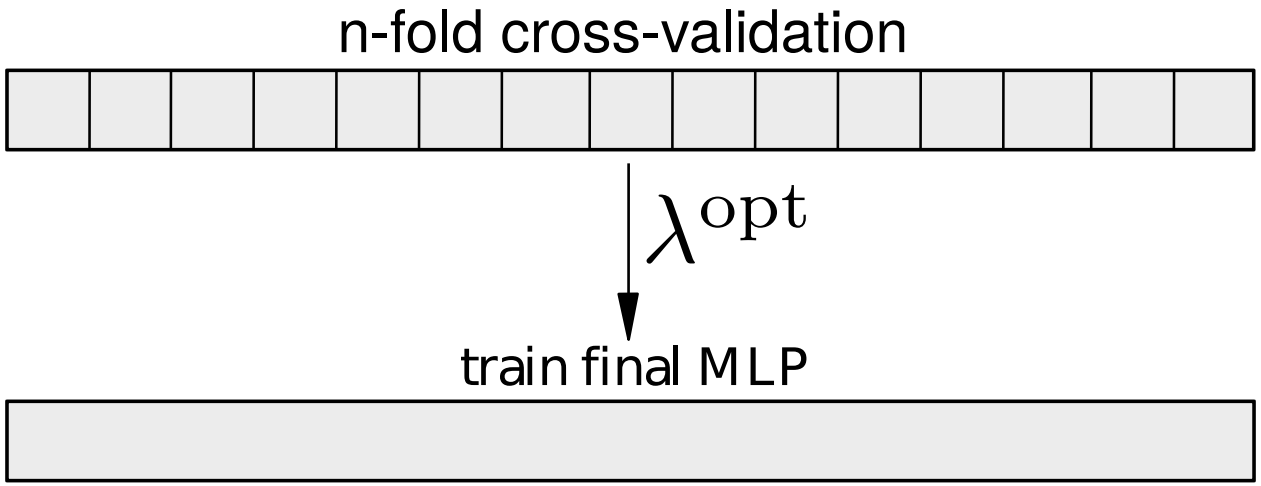
Potential overfitting of λ^{opt} to test set.

n-fold cross-validation for hyperparameter selection

for $\lambda = \lambda_1$ **to** λ_n **do**
  perform n-fold cross-validation on data with regularization λ
end

pick optimal λ^{opt} with minimum $\widehat{E}^G = \langle \widehat{E}^G_k \rangle_{n\text{folds}}$

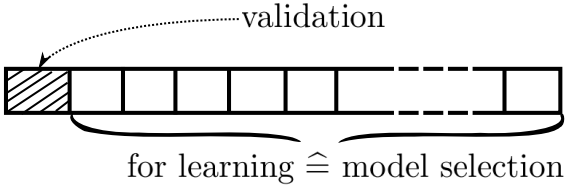
final model: train network on all data with λ^{opt}



Optional: Additional validation set for estimating $\widehat{E}^G$ of final model.

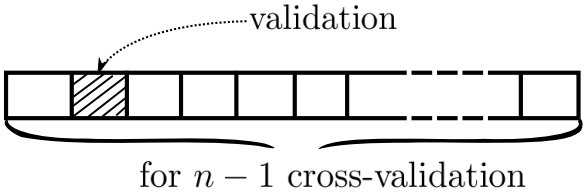
Validation: nested cross-validation

1



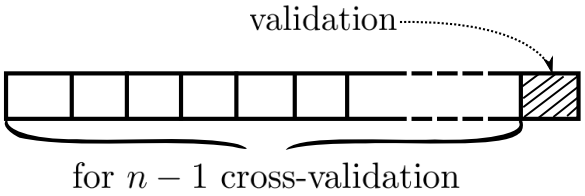
- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

2



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

n



- do n - 1 cross-validation for all values of λ
- train with best λ
- validate with learned model

- $\widehat{E}^G \quad \widehat{=} \quad$ average over all n validation errors

- Never use test data for model selection.
- Always embed the whole selection procedure (including hyper-parameter search) within an n-fold cross-validation run.
- Watch out for validation data "leaking" into the training set.

Still with me?

Gradient methods

Regularization

Validation

Additional Material

- determination of $\underline{\mathbf{g}}$ & $\underline{\mathbf{H}}$ for linear regression (quadr. cost)
- solve non-linear problems
- identify the bias-variance tradeoff
- regularize our model

The setting for our linear regression

Data:

observations	$x \in \mathbb{R}$
label	$y_T \in \mathbb{R}$
Dataset	$\left\{ \left(x^{(1)}, y_T^{(1)}\right), \ldots, \left(x^{(\alpha)}, y_T^{(\alpha)}\right), \ldots, \left(x^{(p)}, y_T^{(p)}\right) \right\}$

Model:
linear neuron:

$$y(x; \underline{\mathbf{w}}) = w_0 + w_1 x = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}$$

with $\underline{\mathbf{x}} := (1, x)^\top \in \mathbb{R}^2$ and $\underline{\mathbf{w}} := (w_0, w_1)^\top \in \mathbb{R}^2$

Cost function:
Quadratic error

$$E^T = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{1}{2} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)} \right)^2$$
$$= \frac{1}{p} \sum_{\alpha=1}^{p} e^{(\alpha)}$$

Optimization:

Cost function:
Quadratic error

$$E^T = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{1}{2} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)} \right)^2$$
$$= \frac{1}{p} \sum_{\alpha=1}^{p} e^{(\alpha)}$$

Optimization:

$$E^T \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$
$$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} = \underline{\mathbf{0}} \qquad \text{(satisfied by extrema)}$$

Calculating the gradient

$$e = \frac{1}{2}\,(y - y_T)^2$$

$$y = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}$$

$$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} = \frac{1}{p}\sum_{\alpha=1}^{p} \frac{\partial e^{(\alpha)}}{\partial \underline{\mathbf{w}}}$$

$$\frac{\partial e}{\partial \underline{\mathbf{w}}} \overset{\text{chain rule}}{=} \frac{\partial e}{\partial y} \cdot \frac{\partial y}{\partial \underline{\mathbf{w}}}$$

$$= \left(y - y_T\right) \cdot \frac{\partial}{\partial \underline{\mathbf{w}}} \underline{\mathbf{w}}^\top \underline{\mathbf{x}}$$

using $\frac{\partial}{\partial \underline{\mathbf{a}}}\left(\underline{\mathbf{a}}^\top \underline{\mathbf{b}}\right) = \underline{\mathbf{b}} = (b_1, b_2, \ldots, b_N)^\top$

$$= \left(\underline{\mathbf{w}}^\top \underline{\mathbf{x}} - y_T\right) \cdot \underline{\mathbf{x}} \quad = \quad \underline{\mathbf{x}}\,\underline{\mathbf{w}}^\top \underline{\mathbf{x}} - \underline{\mathbf{x}}\, y_T$$

$$\frac{\partial E^T}{\partial \underline{\mathbf{w}}} = \frac{1}{p}\sum_{\alpha=1}^{p} \left(\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}\, y_T\right) \quad =: \quad \underline{\mathbf{g}}$$

Finding minima

$$\underline{\mathbf{H}} \equiv \frac{\partial^2 E^T}{\partial \underline{\mathbf{w}}^2} = \begin{pmatrix} \frac{\partial^2 E^T}{\partial w_0^2} & \frac{\partial^2 E^T}{\partial w_0 \partial w_1} & \cdots & \frac{\partial^2 E^T}{\partial w_0 \partial w_N} \\ \frac{\partial^2 E^T}{\partial w_1 \partial w_0} & \frac{\partial^2 E^T}{\partial w_1^2} & \cdots & \frac{\partial^2 E^T}{\partial w_1 \partial w_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 E^T}{\partial w_N \partial w_0} & \frac{\partial^2 E^T}{\partial w_N \partial w_1} & \cdots & \frac{\partial^2 E^T}{\partial w_N^2} \end{pmatrix}$$

with $\underline{\mathbf{g}} = \frac{\partial E^T}{\partial \underline{\mathbf{w}}}$

Finding minima

$$\underline{\mathbf{H}} \equiv \frac{\partial^2 E^T}{\partial \underline{\mathbf{w}}^2} = \begin{pmatrix} \frac{\partial^2 E^T}{\partial w_0^2} & \frac{\partial^2 E^T}{\partial w_0 \partial w_1} & \cdots & \frac{\partial^2 E^T}{\partial w_0 \partial w_N} \\ \frac{\partial^2 E^T}{\partial w_1 \partial w_0} & \frac{\partial^2 E^T}{\partial w_1^2} & \cdots & \frac{\partial^2 E^T}{\partial w_1 \partial w_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 E^T}{\partial w_N \partial w_0} & \frac{\partial^2 E^T}{\partial w_N \partial w_1} & \cdots & \frac{\partial^2 E^T}{\partial w_N^2} \end{pmatrix}$$

with $\underline{\mathbf{g}} = \frac{\partial E^T}{\partial \underline{\mathbf{w}}}$ the i-th <u>row</u> in $\underline{\mathbf{H}}$ resembles $\frac{\partial}{\partial w_i} \underline{\mathbf{g}}^\top$.
Therefore,

$$\underline{\mathbf{H}} := \frac{\partial}{\partial \underline{\mathbf{w}}} \underline{\mathbf{g}}^\top$$

$$\underline{\mathbf{H}} := \frac{\partial}{\partial \underline{\mathbf{w}}} \underline{\mathbf{g}}^\top = \frac{1}{p} \sum_{\alpha=1}^{p} \frac{\partial}{\partial \underline{\mathbf{w}}} \left(\frac{e^{(\alpha)}}{y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial \underline{\mathbf{w}}} \right)^\top$$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} \frac{\partial}{\partial \underline{\mathbf{w}}} \left(\underline{\mathbf{x}}^{(\alpha)} \, \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)} y_T^{(\alpha)} \right)^\top$$

using $\left(\underline{\mathbf{A}} \; \underline{\mathbf{B}} \; \underline{\mathbf{C}}\right)^\top = \underline{\mathbf{C}}^\top \underline{\mathbf{B}}^\top \underline{\mathbf{A}}^\top$ and $\underline{\mathbf{x}}^{(\alpha)\top} \underline{\mathbf{w}} = \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)}$

$$= \frac{1}{p} \sum_{\alpha=1}^{p} \left(\frac{\partial}{\partial \underline{\mathbf{w}}} \, \underline{\mathbf{w}}^\top \underline{\mathbf{x}}^{(\alpha)} \; \underline{\mathbf{x}}^{(\alpha)\top} - \frac{\partial}{\partial \underline{\mathbf{w}}} \Big(\underbrace{\underline{\mathbf{x}}^{(\alpha)} y_T^{(\alpha)}}_{\text{indep. of } \underline{\mathbf{w}}} \Big)^\top \right)$$

using the product rule

$$\underline{\mathbf{H}} = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)} \; \underline{\mathbf{x}}^{(\alpha)\top}$$

For the quadratic error:

$$\underline{\mathbf{g}} = \frac{1}{p}\sum_{\alpha=1}^{p}\left(\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{w}}^{\top}\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)}y_T^{(\alpha)}\right)$$

$$\underline{\mathbf{H}} = \frac{1}{p}\sum_{\alpha=1}^{p}\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{x}}^{(\alpha)\top}$$

For the quadratic error:

$$\underline{\mathbf{g}} = \frac{1}{p}\sum_{\alpha=1}^{p}\left(\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{w}}^{\top}\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)}y_T^{(\alpha)}\right)$$

$$\underline{\mathbf{H}} = \frac{1}{p}\sum_{\alpha=1}^{p}\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{x}}^{(\alpha)\top}$$

The data in matrix form

Let $\underline{\mathbf{X}}$ represent the observations for all training points:

$$\underline{\mathbf{X}} := \begin{pmatrix} | & | & & | & & | \\ \underline{\mathbf{x}}^{(1)} & \underline{\mathbf{x}}^{(2)} & \dots & \underline{\mathbf{x}}^{(\alpha)} & \dots & \underline{\mathbf{x}}^{(p)} \\ | & | & & | & & | \end{pmatrix} \in \mathbb{R}^{2\times p}$$

and let $\underline{\mathbf{y}}_{\text{True}}$ represent the true labels for all training points:

$$\underline{\mathbf{y}}_{\text{True}} := (y_T^{(1)}, \dots, y_T^{(p)}) \in \mathbb{R}^{1\times p}$$

The qudaratic cost (MSE)

$$\underline{\mathbf{X}} \in \mathbb{R}^{2\times p}$$

$$\underline{\mathbf{y}}_{\text{True}} \in \mathbb{R}^{1\times p}$$

$$\underline{\mathbf{H}} := \frac{1}{p}\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top$$

$$\begin{aligned} E^T_{\text{quadr.}} &= \frac{1}{2p}\left\|\underline{\mathbf{w}}^\top\underline{\mathbf{X}} - \underline{\mathbf{y}}_{\text{True}}\right\|_2^2 \\ &= \frac{1}{2p}\underbrace{\left(\underline{\mathbf{w}}^\top\underline{\mathbf{X}} - \underline{\mathbf{y}}_{\text{True}}\right)}_{1\times p}\underbrace{\left(\underline{\mathbf{w}}^\top\underline{\mathbf{X}} - \underline{\mathbf{y}}_{\text{True}}\right)^\top}_{p\times 1} \end{aligned}$$

with $\underline{\mathbf{y}}_{\text{True}} = \underline{\mathbf{w}}^{*\top}\underline{\mathbf{X}}$:

$$\begin{aligned} &= \frac{1}{2p}\left(\underline{\mathbf{w}}^\top\underline{\mathbf{X}} - \mathbf{w}^{*\top}\underline{\mathbf{X}}\right)\left(\underline{\mathbf{w}}^\top\underline{\mathbf{X}} - \mathbf{w}^{*\top}\underline{\mathbf{X}}\right)^\top \\ &= \frac{1}{2p}\left(\underline{\mathbf{w}} - \mathbf{w}^*\right)^\top\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\left(\underline{\mathbf{w}} - \mathbf{w}^*\right) \\ &= \frac{1}{2}\left(\underline{\mathbf{w}} - \mathbf{w}^*\right)^\top\underline{\mathbf{H}}\left(\underline{\mathbf{w}} - \mathbf{w}^*\right) \end{aligned}$$

The gradient

$$\underline{\mathbf{X}} \in \mathbb{R}^{2\times p}$$

$$\underline{\mathbf{y}}_{\mathsf{True}} \in \mathbb{R}^{1\times p}$$

$$\underline{\mathbf{H}} := \frac{1}{p}\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top$$

$$\begin{aligned}
\underline{\mathbf{g}} &= \frac{1}{p}\sum_{\alpha=1}^{p}\left(\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{w}}^\top\underline{\mathbf{x}}^{(\alpha)} - \underline{\mathbf{x}}^{(\alpha)}y_T^{(\alpha)}\right)\\
&= \frac{1}{p}\sum_{\alpha=1}^{p}\left(\underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{x}}^{(\alpha)\top}\underline{\mathbf{w}} - \underline{\mathbf{x}}^{(\alpha)}y_T^{(\alpha)}\right)\\
&= \frac{1}{p}\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\mathsf{True}}^\top\right) = \underline{\mathbf{H}}\,\underline{\mathbf{w}} - \frac{1}{p}\underline{\mathbf{X}}\underline{\mathbf{y}}_{\mathsf{True}}^\top
\end{aligned}$$

The gradient

$$\underline{\mathbf{g}} = \frac{1}{p}\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top\right)$$

stationary point:

$$\underline{\mathbf{g}} \stackrel{!}{=} \underline{\mathbf{0}} \iff \underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\underline{\mathbf{w}}^* = \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\right)}_{\substack{\text{assumes}\\ \text{invertibility}}}^{-1}\underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

$\underline{\mathbf{w}}^*$ is a global minimum because...?

The gradient

stationary point:

$$\underline{\mathbf{g}} \stackrel{!}{=} \underline{\mathbf{0}} \iff \underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top \underline{\mathbf{w}}^* = \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\right)}_{\substack{\text{assumes}\\ \text{invertibility}}}^{-1} \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

$\underline{\mathbf{w}}^*$ is a global minimum because

1. it is a unique solution of $\underline{\mathbf{g}} = \underline{\mathbf{0}}$ and
2. $$\underline{\mathbf{H}} = \frac{1}{p}\sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{x}}^{(\alpha)\top} = \frac{1}{p}\,\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top = \underline{\mathbf{C}}$$

   where $\underline{\mathbf{C}}$ is the *empirical covariance matrix* of centered data which is positive definite for invertible $\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top$.

The gradient

stationary point:

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top\right)^{-1}}_{\substack{\text{assumes}\\ \text{invertibility}}} \underline{\mathbf{X}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

$\underline{\mathbf{w}}^*$ is a global minimum because

1. it is a unique solution of $\underline{\mathbf{g}} = \underline{\mathbf{0}}$ and
2. $$\underline{\mathbf{H}} = \frac{1}{p}\sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}\,\underline{\mathbf{x}}^{(\alpha)\top} = \frac{1}{p}\,\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top = \underline{\mathbf{C}}$$

   where $\underline{\mathbf{C}}$ is the *empirical covariance matrix* of centered data which is positive definite for invertible $\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top$.

positive definiteness:

$$\forall \underline{\mathbf{z}} \in \mathbb{R}^2: \quad \underline{\mathbf{z}}^\top \underline{\mathbf{H}}\,\underline{\mathbf{z}} = \frac{1}{p}\,\underline{\mathbf{z}}^\top \underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top \underline{\mathbf{z}} = \frac{1}{p}\left\|\underline{\mathbf{X}}^\top \underline{\mathbf{z}}\right\|_2^2 > 0 \quad \text{iff. } \underline{\mathbf{z}} \neq \underline{\mathbf{0}}$$

L_2 regularization cost:

$$E^R_{[\underline{\mathbf{w}}]} := \frac{1}{2} \sum_{(i,j,v',v)} \left(\mathrm{w}^{v'v}_{ij}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}}$$

$$\frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} = \ldots?$$

L_2 regularization cost:

$$E^R_{[\underline{\mathbf{w}}]} := \frac{1}{2} \sum_{(i,j,v',v)} \left(\mathrm{w}_{ij}^{v'v}\right)^2 = \frac{1}{2}\|\underline{\mathbf{w}}\|_2^2 = \frac{1}{2}\underline{\mathbf{w}}^\top \underline{\mathbf{w}}$$

$$\frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} = \frac{1}{\cancel{2}} \cdot \cancel{2} \cdot \underline{\mathbf{w}} = \underline{\mathbf{w}}$$

The regularized cost function:

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training} \\ \text{error} \\ \text{e.g. quadratic cost}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization} \\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

where

- $E^R \mathrel{\widehat{=}}$ prior knowledge of solution
- λ : regularization parameter

Gradient:

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training error} \\ \text{e.g. quadratic cost}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization} \\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

$$\underline{\mathbf{g}}^R := ...?$$

Gradient:

$$R_{[\underline{\mathbf{w}}]} = \underbrace{E^T_{[\underline{\mathbf{w}}]}}_{\substack{\text{training}\\ \text{error}\\ \text{e.g. quadratic cost}}} + \underbrace{\lambda E^R_{[\underline{\mathbf{w}}]}}_{\substack{\text{regularization}\\ \text{term}}} \stackrel{!}{=} \min_{\underline{\mathbf{w}}}$$

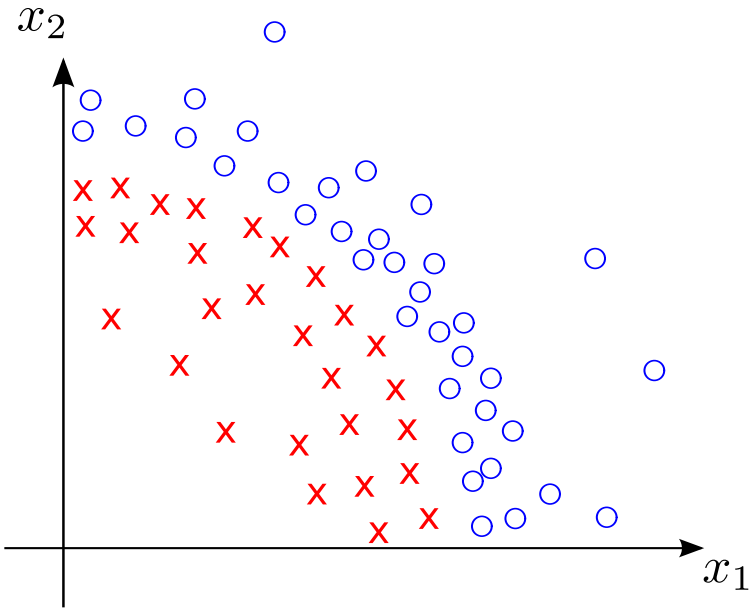
$$\begin{aligned}
\underline{\mathbf{g}}^R := \frac{\partial}{\partial \underline{\mathbf{w}}} R_{[\underline{\mathbf{w}}]} &= \frac{\partial}{\partial \underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]} + \frac{\partial}{\partial \underline{\mathbf{w}}} E^R_{[\underline{\mathbf{w}}]} \\
&= \frac{1}{p}\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}^\top_{\text{True}}\right) + \underline{\mathbf{I}}_N\,\lambda\,\underline{\mathbf{w}}
\end{aligned}$$

$$\iff \quad \underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top \underline{\mathbf{w}} - \underline{\mathbf{X}}\,\underline{\mathbf{y}}^\top_{\text{True}} + p\lambda\,\underline{\mathbf{I}}_N\,\underline{\mathbf{w}} \stackrel{!}{=} \underline{\mathbf{0}}$$

$$\underline{\mathbf{w}}^* = \underbrace{\left(\underline{\mathbf{X}}\,\underline{\mathbf{X}}^\top + p\lambda\,\underline{\mathbf{I}}_N\right)}_{\substack{\text{positive definite and}\\ \text{therefore invertible}}}{}^{-1} \underline{\mathbf{X}}\,\underline{\mathbf{y}}^\top_{\text{True}}$$

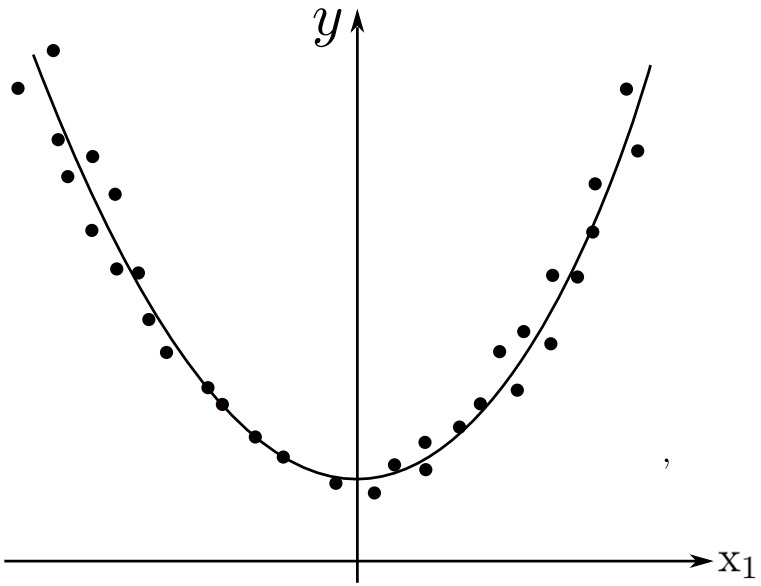
A non-linearly separable problem

Q: *Find a perfect linear separation between the classes using only a connectionist neuron.*



A non-linearly separable problem

Q: *Fit this parabola using only a connectionist neuron.*

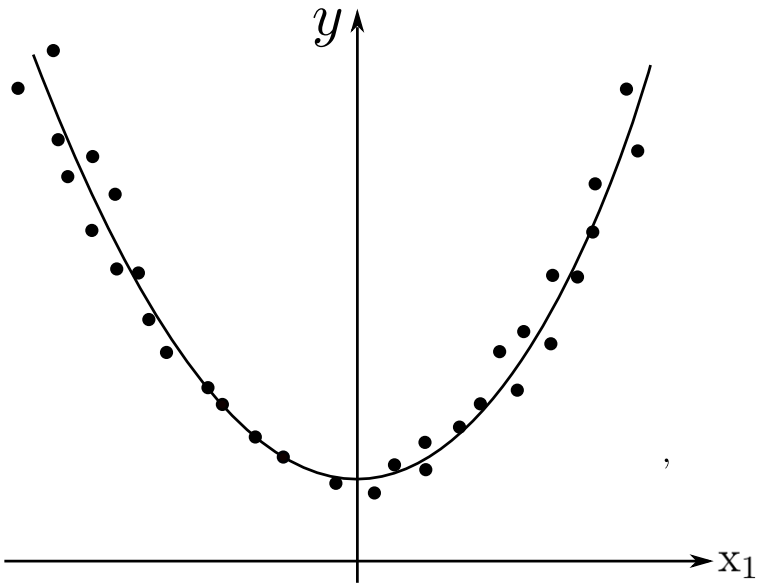


Q: *How would you characterize any parabolic (quadratic) function?*

$$y(x) =$$

A non-linearly separable problem

Q: *Fit this parabola using only a connectionist neuron.*



Q: *How would you characterize any parabolic (quadratic) function?*

$$y(x) = a \cdot x^2 + b \cdot x + c = w_2 \cdot x^2 + w_1 \cdot x + w_0$$

Nonlinear basis function

Feature transformation ($\underline{\mathbf{x}} \in \mathbb{R}^N$):

$$\underline{\phi} : \underline{\mathbf{x}} \mapsto \underline{\phi}(\underline{\mathbf{x}})$$

Expanding our feature space using *Monomials* of highest degree k (i.e. polynomial expansion):

$$\phi_i(\underline{\mathbf{x}}) \in \{\ 1, x_1\ ,\ x_2, \ldots, x_N, x_1^2\ ,\ x_1 x_2, \ldots, x_1 x_N, \ldots,\ x_N^2,\ \ldots \\ x_1 x_N^{k-1}\ ,\ x_1^k, \ldots, x_N^k\}$$

$$\phi_i(\underline{\mathbf{x}}) \in \left\{ \prod_{j=1}^{N} x_j^{a_j} \quad : a_j \in \mathbb{N}_0 \ \text{ with } \ 0 \leq \sum_{j=1}^{N} a_j \leq k \right\}$$

where $i = 1, \ldots d$.
Example: $k = 9, N = 2 \quad \Rightarrow \quad d = 55$

Preprocessing: Sphering

Large $\|\underline{\mathbf{x}}\|$ can lead to very large $\phi_i(\underline{\mathbf{x}})$.
We want to decorrelate the transformed data and standardize it (keep variance of $x_i = 1$) For every point $\alpha = 1, \ldots, p$ from the training set:

$$\underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{\Lambda}}^{-\frac{1}{2}} \underline{\mathbf{E}}^{\top} \underline{\mathbf{x}}^{(\alpha)}_{\text{centered}}.$$

Here

$\underline{\mathbf{x}}^{(\alpha)}_{\text{centered}} = \underline{\mathbf{x}}^{(\alpha)} - \langle \underline{\mathbf{x}} \rangle \qquad \langle \underline{\mathbf{x}} \rangle = \frac{1}{p} \sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}$,
$\underline{\mathbf{E}} = (\underline{\mathbf{e}}_1, \ldots, \underline{\mathbf{e}}_N)$ is the eigenvector matrix and
$\underline{\mathbf{\Lambda}} = \operatorname{diag}(\lambda_1, \ldots, \lambda_N)$ is the eigenvalue matrix for the eigendecomposition

$$\underline{\mathbf{C}}\, \underline{\mathbf{e}}_i = \lambda_i \underline{\mathbf{e}}_i$$

of the covariance matrix $\underline{\mathbf{C}}$ with $C_{ij} = \frac{1}{p} \sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{centered},i}\, x^{(\alpha)}_{\text{centered},j}$.

Preprocessing: Sphering

Examining the properties of the data after sphering:

$$\frac{1}{p}\sum_{\alpha=1}^{p} \underline{\mathbf{x}}^{(\alpha)}_{\text{sphered}} = \underline{\mathbf{0}} \quad \text{(centered)},$$

$$\frac{1}{p}\sum_{\alpha=1}^{p} (x^{(\alpha)}_{\text{sphered,i}})^2 = 1 \quad \text{(unit variance)},$$

$$\frac{1}{p}\sum_{\alpha=1}^{p} x^{(\alpha)}_{\text{sphered},i}\, x^{(\alpha)}_{\text{sphered},j} = 0 \quad \text{(zero covariance)},$$

Important: Reuse the same pre-processing (centering, sphering) on validation set. Also apply to test-set in case of a 3-way split.

Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Linear neuron for regression: Analytical solution

Data:

transformed input	$\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$
label	$y_T^{(\alpha)} \in \mathbb{R}$
with	$\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \underline{\mathbf{w}}^\top \underline{\phi} = \sum_{i=1}^{d} w_i \phi_i(\underline{\mathbf{x}})$$

quadratic cost function, L_2 regularization.

Linear neuron for regression: Analytical solution

Data:

transformed input $\underline{\phi}^{(\alpha)} := \underline{\phi}(\underline{\mathbf{x}}^{(\alpha)}) = \left(\phi_1(\underline{\mathbf{x}}^{(\alpha)}), \phi_2(\underline{\mathbf{x}}^{(\alpha)}), \ldots, \phi_d(\underline{\mathbf{x}}^{(\alpha)})\right)^\top$

label $y_T^{(\alpha)} \in \mathbb{R}$

with $\alpha = 1, \ldots, p$ and $\underline{\mathbf{x}}$ has been centered and sphered.

Model:

$$y(\underline{\phi}; \underline{\mathbf{w}}) = \underline{\mathbf{w}}^\top \underline{\phi} = \sum_{i=1}^{d} w_i \phi_i(\underline{\mathbf{x}})$$

quadratic cost function, L_2 regularization. Reuse solution for regularized linear regression:

$$\underline{\mathbf{w}}^* = \left(\underline{\mathbf{\Phi}}\,\underline{\mathbf{\Phi}}^\top + \lambda \underline{\mathbf{I}}_d\right)^{-1} \underline{\mathbf{\Phi}}\,\underline{\mathbf{y}}_{\text{True}}^\top$$

where $\underline{\mathbf{\Phi}} = \left(\underline{\phi}^{(1)}, \ldots, \underline{\phi}^{(\alpha)}, \ldots, \underline{\phi}^{(p)}\right) \in \mathbb{R}^{d,p}$